

The Unbearable Notation of Logic

Florian Lengyel

September 26, 2026

Abstract

Standard presentations of first-order syntax and semantics routinely use similar brackets, superscripts, and substitution expressions for several differently typed operations. Fix a first-order language $\mathcal{L}$ with functional signature Σ , sets X, Y of variables, and an $\mathcal{L}$ -structure with domain A . Four relevant types are:

$$X \rightarrow T_\Sigma Y, \quad X \rightarrow A, \quad T_\Sigma X \rightarrow A, \quad \text{Form}_\mathcal{L}(X) \times A^X \rightarrow \mathbf{2}.$$

Here A^X is the set of functions $X \rightarrow A$, and $\mathbf{2} = \{\perp, \top\}$ is the set of truth values. The four displayed maps represent syntactic substitution, assignment, term evaluation under an assignment, and satisfaction. Readers must ordinarily infer from convention which operation an expression denotes.

This note presents two complementary descriptions of the operations. The categorical description organizes terms into a monad, substitutions into Kleisli arrows, and functional structures into Eilenberg–Moore algebras. The @-calculus uses prefix @ for term evaluation and postfix @ for satisfaction; it is the working notation for calculations. The categorical notation records compositional structure, while the @-notation performs the corresponding term-evaluation and satisfaction calculations. Adjoining names for the elements of a structure changes the term functor from $T_\Sigma X$ to $T_\Sigma(A + X)$ and turns assignments into substitutions by ground terms. A finite-occurrence construction of Došen–Petric records how generalized composition routes repeated formula-pattern occurrences. Functorial semantics then extends the categorical description to internal models and, for geometric theories, to models as generalized points of a classifying topos. An appendix separates expressions, codes, and numeral-valued terms in Feferman’s arithmetization conventions.

First-order syntax and semantics involve terms, syntactic substitutions, assignments, term evaluation, formulas, and satisfaction. Textbook notation often leaves the types distinguishing those six ingredients to convention. The term-monad description below explains how the ingredients compose; the @-calculus makes their substitution laws concise enough for routine calculation. The categorical diagrams state the structural laws, and the @-identities state their calculational forms.

The note uses the familiar inductive definitions of terms, formulas, evaluation, and satisfaction. Categorical notation records their compositional laws, and the @-calculus supplies their working calculation notation. A later section adds the Došen–Petric occurrence maps, which record how formula-pattern substitution routes the individual occurrences of the outer pattern’s formal arguments.

1 Categorical vocabulary

A **category** consists of objects and arrows between them. For every object X there is an identity arrow $1_X : X \rightarrow X$, and composable arrows $f : X \rightarrow Y$ and $g : Y \rightarrow Z$ have a composite $g \circ f : X \rightarrow Z$. Composition is associative, and the identity laws are $f \circ 1_X = f = 1_Y \circ f$. The category **Set** has sets as objects and functions as arrows.

A **functor** $F : \mathcal{C} \rightarrow \mathcal{D}$ sends objects to objects and arrows to arrows while preserving identities and composition. An **endofunctor** is a functor from a category to itself.

Given functors $F, G : \mathcal{C} \rightarrow \mathcal{D}$, a **natural transformation** $\theta : F \Rightarrow G$ assigns an arrow $\theta_X : FX \rightarrow GX$ to every object X so that, for every $u : X \rightarrow Y$, the following square commutes:

$$\begin{array}{ccc} FX & \xrightarrow{\theta_X} & GX \\ F(u) \downarrow & & \downarrow G(u) \\ FY & \xrightarrow{\theta_Y} & GY \end{array}$$

Equivalently, $G(u) \circ \theta_X = \theta_Y \circ F(u)$: applying θ is compatible with changing X by u . The unit of the term monad, defined in Section 3, is a natural transformation. Its naturality says that mapping variable leaves along $u : X \rightarrow Y$ commutes with regarding those variables as terms.

A **monad** on a category $\mathcal{C}$ is an endofunctor $T : \mathcal{C} \rightarrow \mathcal{C}$ together with natural transformations $\eta : 1_{\mathcal{C}} \Rightarrow T$ and $\mu : T^2 \Rightarrow T$, called the unit and multiplication. Here $1_{\mathcal{C}}$ is the identity functor, which fixes every object and arrow, while $T^2 = T \circ T$ and $T^3 = T \circ T \circ T$ are iterated applications of T . The monad laws are typed by the following commutative diagrams. For every object X , the two unit triangles are

$$\begin{array}{ccc} TX & \xrightarrow{\eta_{TX}} & T^2X \\ & \searrow 1_{TX} & \downarrow \mu_X \\ & & TX \end{array} \quad \begin{array}{ccc} TX & \xrightarrow{T(\eta_X)} & T^2X \\ & \searrow 1_{TX} & \downarrow \mu_X \\ & & TX, \end{array}$$

and associativity is the square

$$\begin{array}{ccc} T^3X & \xrightarrow{T(\mu_X)} & T^2X \\ \mu_{TX} \downarrow & & \downarrow \mu_X \\ T^2X & \xrightarrow{\mu_X} & TX. \end{array}$$

For the term monad below, T forms terms, η regards a variable as a term, and μ removes one formal term layer. Section 3 defines this operation recursively and computes it on an explicit term of terms.

The **Kleisli category** $\mathbf{Kl}(T)$ of a monad T has the same objects as $\mathcal{C}$. An arrow from X to Y in $\mathbf{Kl}(T)$ is an ordinary arrow $X \rightarrow TY$ in $\mathcal{C}$. The identity on X is $\eta_X : X \rightarrow TX$. If $\sigma : X \rightarrow TY$ and $\tau : Y \rightarrow TZ$, their Kleisli composite is $\tau \star \sigma = \mu_Z \circ T(\tau) \circ \sigma : X \rightarrow TZ$. For the term monad, Kleisli arrows are precisely simultaneous syntactic substitutions.

An **Eilenberg–Moore algebra** for a monad T is an object A equipped with an arrow $a : TA \rightarrow A$ for which the following unit triangle and multiplication square commute:

$$\begin{array}{ccc} A & \xrightarrow{\eta_A} & TA \\ & \searrow 1_A & \downarrow a \\ & & A \end{array} \quad \begin{array}{ccc} T^2A & \xrightarrow{T(a)} & TA \\ \mu_A \downarrow & & \downarrow a \\ TA & \xrightarrow{a} & A. \end{array}$$

It specifies a coherent way to interpret or evaluate the formal expressions collected by T . A homomorphism $h : (A, a) \rightarrow (B, b)$ of such algebras is an arrow $h : A \rightarrow B$ for which the following square commutes:

$$\begin{array}{ccc}
TA & \xrightarrow{T(h)} & TB \\
a \downarrow & & \downarrow b \\
A & \xrightarrow{h} & B
\end{array}$$

The algebras and their homomorphisms form the **Eilenberg–Moore category** $\mathcal{C}^T$. For the term monad, its objects are the ordinary algebras interpreting the function symbols of a signature.

For monads T, S on the same category, a **monad morphism** $j : T \Rightarrow S$ is a natural transformation for which the following unit triangle and multiplication rectangle commute for every X :

$$\begin{array}{ccccc}
X & \xrightarrow{\eta_X^T} & TX & & T^2X & \xrightarrow{j_{TX}} & STX & \xrightarrow{S(j_X)} & S^2X \\
& \searrow \eta_X^S & \downarrow j_X & & \mu_X^T \downarrow & & & & \downarrow \mu_X^S \\
& & SX & & TX & \xrightarrow{j_X} & SX
\end{array}$$

Thus j translates T -expressions into S -expressions while preserving variables and substitution.

2 Languages and their functional signatures

Let $\mathcal{L}$ be a single-sorted first-order language. Separate its nonlogical symbols into:

- a finitary functional signature Σ , including constant symbols as nullary function symbols; and
- a family $\mathcal{R}$ of relation symbols with specified arities.

The term construction depends only on Σ . Relation symbols enter later, when formulas and satisfaction are defined.

For each set X of variables, let $T_\Sigma X$ be the set of Σ -terms whose variables belong to X . Thus $T_\Sigma X$ is generated by variables $x \in X$ and expressions $f(t_1, \dots, t_n)$, where $f \in \Sigma$ has arity n and each t_i belongs to $T_\Sigma X$.

A map $u : X \rightarrow Y$ sends each variable leaf x to the variable leaf $u(x)$ and induces $T_\Sigma u : T_\Sigma X \rightarrow T_\Sigma Y$, recursively defined by

$$T_\Sigma u(\eta_X(x)) = \eta_Y(u(x)), \quad T_\Sigma u(f(t_1, \dots, t_n)) = f(T_\Sigma u(t_1), \dots, T_\Sigma u(t_n)).$$

For every $v : Y \rightarrow Z$, structural induction yields $T_\Sigma(1_X) = 1_{T_\Sigma X}$ and $T_\Sigma(v \circ u) = T_\Sigma v \circ T_\Sigma u$. Hence T_Σ is an endofunctor on **Set**.

3 The term monad

The endofunctor T_Σ has a canonical monad structure. Its unit $\eta_X : X \rightarrow T_\Sigma X$ sends a variable to the corresponding variable term. To define its multiplication, first observe that $T_\Sigma^2 X = T_\Sigma(T_\Sigma X)$. An element of this set is an outer Σ -term whose variable labels are themselves elements of $T_\Sigma X$. If $p \in T_\Sigma X$, then $\eta_{T_\Sigma X}(p)$ is the outer variable labelled by the entire inner term p .

The multiplication $\mu_X : T_\Sigma(T_\Sigma X) \rightarrow T_\Sigma X$ is defined by structural recursion:

$$\begin{aligned}
\mu_X(\eta_{T_\Sigma X}(p)) &= p, & p &\in T_\Sigma X, \\
\mu_X(f(U_1, \dots, U_n)) &= f(\mu_X(U_1), \dots, \mu_X(U_n)), & f &\in \Sigma \text{ of arity } n.
\end{aligned}$$

Every term has exactly one outermost constructor: it is either a variable term or an expression $f(U_1, \dots, U_n)$. The two clauses therefore assign exactly one value to every element of $T_\Sigma(T_\Sigma X)$, and structural recursion yields a function μ_X with the displayed type. The operation is often called *flattening*: it replaces every outer variable by the inner term labelling that variable and preserves every operation symbol.

Example: flattening a term of terms. Use a signature containing the nullary symbol 1, the unary successor symbol S , and the binary symbols $+$ and $\cdot$, and let $X = \{x, y\}$. In $T_\Sigma X$, put $p = x + 1$, $q = S(y)$, and $r = x \cdot y$, and abbreviate

$$\hat{p} = \eta_{T_\Sigma X}(p), \quad \hat{q} = \eta_{T_\Sigma X}(q), \quad \hat{r} = \eta_{T_\Sigma X}(r).$$

The hats are outer variable terms labelled by the entire terms p, q, r . Consequently,

$$U = \hat{p} + (\hat{q} \cdot \hat{r}) \in T_\Sigma(T_\Sigma X).$$

The recursive clauses calculate its value one constructor at a time:

$$\begin{aligned} \mu_X(U) &= \mu_X(\hat{p}) + (\mu_X(\hat{q}) \cdot \mu_X(\hat{r})) \\ &= p + (q \cdot r) \\ &= (x + 1) + (S(y) \cdot (x \cdot y)). \end{aligned}$$

The first recursive clause removes each hat; the second rebuilds the displayed sum and product around the resulting inner terms. Thus “flatten” names the fully specified recursive operation above.

The unit law $\mu_X \circ \eta_{T_\Sigma X} = 1_{T_\Sigma X}$ removes an outer variable labelled by a whole term. The other unit law $\mu_X \circ T_\Sigma(\eta_X) = 1_{T_\Sigma X}$ replaces every variable by its corresponding variable term. The associativity square says that a three-layer term yields the same ordinary term whether the inner two layers or the outer two layers are reduced first. Structural induction proves both unit laws, associativity, and the naturality of η and μ . For example, if $A = \hat{p} + \hat{q}$ and $B = \hat{r}$ are elements of $T_\Sigma^2 X$, then $W = \eta_{T_\Sigma^2 X}(A) \cdot \eta_{T_\Sigma^2 X}(B)$ belongs to $T_\Sigma^3 X$. Put $\widehat{p + q} = \eta_{T_\Sigma X}(p + q)$. The two paths around the associativity square are the explicit calculations

$$\begin{aligned} (\mu_X \circ T_\Sigma \mu_X)(W) &= \mu_X(\widehat{p + q} \cdot \hat{r}) = (p + q) \cdot r, \\ (\mu_X \circ \mu_{T_\Sigma X})(W) &= \mu_X(A \cdot B) = (p + q) \cdot r. \end{aligned}$$

4 Syntactic substitution as Kleisli composition

A simultaneous substitution from a variable set X into terms over a variable set Y is a family $(s_x)_{x \in X}$, with $s_x \in T_\Sigma Y$. Such a family supplies exactly one term s_x for every $x \in X$; consequently, the rule $x \mapsto s_x$ is a total, single-valued function. Denote it by

$$\sigma : X \longrightarrow T_\Sigma Y, \quad \sigma(x) = s_x.$$

For the familiar finite notation, let V be a variable set, let $x_1, \dots, x_n \in V$ be pairwise distinct, and choose $s_1, \dots, s_n \in T_\Sigma V$. Define $\sigma = [x_1 := s_1, \dots, x_n := s_n]$ by

$$\sigma(z) = \begin{cases} s_i, & z = x_i \text{ for the unique } i \in \{1, \dots, n\}, \\ \eta_V(z), & z \notin \{x_1, \dots, x_n\}. \end{cases}$$

Pairwise distinctness makes the first clause single-valued. The two cases exhaust V , and every displayed value belongs to $T_\Sigma V$. Hence the display defines the function $\sigma : V \rightarrow T_\Sigma V$. The substitution specified by the empty list is η_V ; a one-element list denotes ordinary single-variable substitution.

The term Σ -algebra $T_\Sigma X$ carries the formal operations $f_X(t_1, \dots, t_n) = f(t_1, \dots, t_n)$. A function between two term algebras is a Σ -algebra homomorphism when it preserves all these operations.

Unique extension theorem. Every simultaneous substitution $\sigma : X \rightarrow T_\Sigma Y$ has a unique Σ -algebra homomorphism $\sigma^* : T_\Sigma X \rightarrow T_\Sigma Y$ satisfying

$$\begin{aligned}\sigma^*(\eta_X(x)) &= \sigma(x), \\ \sigma^*(f(t_1, \dots, t_n)) &= f(\sigma^*(t_1), \dots, \sigma^*(t_n)).\end{aligned}$$

The first clause supplies the value on a variable term. The second supplies the value on a composite term from the values already assigned to its immediate subterms. Structural recursion proves existence. For uniqueness, let $h : T_\Sigma X \rightarrow T_\Sigma Y$ satisfy the same clauses. The variable clause yields $h(\eta_X(x)) = \sigma^*(\eta_X(x))$. If equality holds on $t_1, \dots, t_n$, preservation of f yields equality on $f(t_1, \dots, t_n)$. Structural induction now proves $h = \sigma^*$.

The extension property is typed by the triangle

$$\begin{array}{ccc} X & \xrightarrow{\eta_X} & T_\Sigma X \\ & \searrow \sigma & \downarrow \exists! \sigma^* \\ & & T_\Sigma Y. \end{array}$$

The dashed arrow is required to be a Σ -algebra homomorphism. Its preservation of an n -ary symbol f is the commutativity of

$$\begin{array}{ccc} (T_\Sigma X)^n & \xrightarrow{f_X} & T_\Sigma X \\ (\sigma^*)^n \downarrow & & \downarrow \sigma^* \\ (T_\Sigma Y)^n & \xrightarrow{f_Y} & T_\Sigma Y. \end{array}$$

The unique extension is also given by the outer path in the commutative triangle

$$\begin{array}{ccc} T_\Sigma X & \xrightarrow{T_\Sigma \sigma} & T_\Sigma^2 Y \\ & \searrow \sigma^* & \downarrow \mu_Y \\ & & T_\Sigma Y. \end{array}$$

Indeed, $\mu_Y \circ T_\Sigma \sigma$ sends a variable term $\eta_X(x)$ to $\sigma(x)$, preserves every operation symbol, and hence equals σ^* by uniqueness. Taking $X = T_\Sigma Y$ and $\sigma = 1_{T_\Sigma Y}$ yields $\mu_Y = (1_{T_\Sigma Y})^*$. Monad multiplication is therefore the unique homomorphic extension that substitutes each term for the outer variable carrying that term as its label.

Example: a simultaneous substitution. Let $X = \{x, y\}$, $Y = \{u, v\}$, and define the function $\sigma : X \rightarrow T_\Sigma Y$ by $\sigma(x) = u + 1$ and $\sigma(y) = u \cdot v$. For $t = x \cdot (y + 1)$, the recursive clauses yield

$$\sigma^*(t) = (u + 1) \cdot ((u \cdot v) + 1).$$

The factorization through a term of terms is visible before multiplication:

$$T_\Sigma \sigma(t) = \eta_{T_\Sigma Y}(u + 1) \cdot (\eta_{T_\Sigma Y}(u \cdot v) + 1) \in T_\Sigma(T_\Sigma Y).$$

Applying μ_Y replaces the two displayed outer variables by their term labels. The word “simultaneous” means that the terms $\sigma(x)$ and $\sigma(y)$ are inserted in one operation. For example, $[x := y, y := S(y)]^*(x + y) = y + S(y)$, whereas first replacing x by y and then replacing y by $S(y)$ yields $S(y) + S(y)$.

If $\sigma : X \rightarrow T_\Sigma Y$ and $\tau : Y \rightarrow T_\Sigma Z$, then their Kleisli composite is the outer path in

$$X \xrightarrow{\sigma} T_\Sigma Y \xrightarrow{T_\Sigma \tau} T_\Sigma^2 Z \xrightarrow{\mu_Z} T_\Sigma Z.$$

$\tau \star \sigma$

Let $v : Z \rightarrow T_\Sigma W$. The associativity of iterated syntactic substitution is associativity of composition in the Kleisli category: the following square commutes in $\mathbf{Kl}(T_\Sigma)$, where each displayed arrow is a Kleisli arrow:

$$\begin{array}{ccc} X & \xrightarrow{\tau \star \sigma} & Z \\ \sigma \downarrow & & \downarrow v \\ Y & \xrightarrow{v \star \tau} & W. \end{array}$$

The upper-right path is $v \star (\tau \star \sigma)$, and the lower-left path is $(v \star \tau) \star \sigma$.

Ordinary replacement of a single variable is a special case. Given $s \in T_\Sigma X$ and $x \in X$, define $[x := s] : X \rightarrow T_\Sigma X$ by

$$[x := s](y) = \begin{cases} s, & y = x, \\ \eta_X(y), & y \neq x. \end{cases}$$

Then $t[x := s]$ is the value of the Kleisli extension $[x := s]^*$ at t .

5 Structures as algebras for the term monad

A Σ -algebra on a set A assigns an operation $f^A : A^n \rightarrow A$ to every n -ary function symbol $f \in \Sigma$. Equivalently, it is an Eilenberg–Moore algebra for the term monad: a map $a : T_\Sigma A \rightarrow A$ satisfying the unit triangle and multiplication square displayed in Section 1, with $T = T_\Sigma$.

The map a evaluates a term whose variables have already been replaced by elements of A . The unit law says that an element used as a variable evaluates to itself. The multiplication law says that evaluating a term of terms in one step agrees with first evaluating the inner terms and then the outer term.

For a language containing relation symbols, this algebra is only the functional reduct of an $\mathcal{L}$ -structure, obtained by forgetting the interpretations of the relation symbols.

An $\mathcal{L}$ -structure $\mathcal{A}$ consists of:

- a nonempty set A ;
- a T_Σ -algebra $a : T_\Sigma A \rightarrow A$; and
- a subset $R^A \subseteq A^n$ for every n -ary relation symbol $R \in \mathcal{R}$.

Logical equality is interpreted by the diagonal $\Delta_A \subseteq A \times A$.

6 Assignments and evaluation

Let X be a set of variables. An assignment in $\mathcal{A}$ is a map $\alpha : X \rightarrow A$.

Term evaluation under α is the outer path in the diagram

$$T_\Sigma X \xrightarrow{T_\Sigma \alpha} T_\Sigma A \xrightarrow{a} A.$$

$$\underbrace{\hspace{10em}}_{\llbracket - \rrbracket_\alpha^A}$$

The evaluation map $\llbracket - \rrbracket_\alpha^A$ is a homomorphism from the term Eilenberg–Moore algebra $(T_\Sigma X, \mu_X)$ to (A, a) ; that assertion has the typed form

$$\begin{array}{ccc} T_\Sigma^2 X & \xrightarrow{T_\Sigma(\llbracket - \rrbracket_\alpha^A)} & T_\Sigma A \\ \mu_X \downarrow & & \downarrow a \\ T_\Sigma X & \xrightarrow{\llbracket - \rrbracket_\alpha^A} & A. \end{array}$$

The evaluator extends the assignment: the triangle

$$\begin{array}{ccc} X & \xrightarrow{\eta_X} & T_\Sigma X \\ & \searrow \alpha & \downarrow \llbracket - \rrbracket_\alpha^A \\ & & A \end{array}$$

commutes. Thus $\llbracket x \rrbracket_\alpha^A = \alpha(x)$, and

$$\llbracket f(t_1, \dots, t_n) \rrbracket_\alpha^A = f^A(\llbracket t_1 \rrbracket_\alpha^A, \dots, \llbracket t_n \rrbracket_\alpha^A).$$

The same structural-recursion and structural-induction argument used for σ^* shows that this evaluator is the unique Σ -algebra homomorphism extending α . Because this extension property holds for every Σ -algebra (A, a) and every assignment $\alpha : X \rightarrow A$, $(T_\Sigma X, \mu_X)$ is called the **free** Σ -algebra on X .

For calculations, write the assignment as $@ : X \rightarrow A$ and put $@t := \llbracket t \rrbracket_@^A$. Thus $@x = @ (x)$ and $@f(t_1, \dots, t_n) = f^A(@t_1, \dots, @t_n)$. When the structure must be shown, write $@_A t$.

Now let $\sigma : X \rightarrow T_\Sigma Y$ be a syntactic substitution and let $\beta : Y \rightarrow A$ be an assignment. Substitution followed by evaluation induces the assignment $\beta \bullet \sigma : X \rightarrow A$, shown as the outer path in

$$X \xrightarrow{\sigma} T_\Sigma Y \xrightarrow{T_\Sigma \beta} T_\Sigma A \xrightarrow{a} A.$$

$$\underbrace{\hspace{10em}}_{\beta \bullet \sigma}$$

For each $x \in X$, the value $(\beta \bullet \sigma)(x)$ is the value under β of the term $\sigma(x)$.

Example: evaluating a simultaneous substitution. Continue the earlier example with $X = \{x, y\}$, $Y = \{u, v\}$, $\sigma(x) = u + 1$, $\sigma(y) = u \cdot v$, and $t = x \cdot (y + 1)$. In the arithmetic structure on $\mathbb{N}$, let $@ : Y \rightarrow \mathbb{N}$ satisfy $@(u) = 2$ and $@(v) = 3$. Then $(@ \bullet \sigma)(x) = 3$ and $(@ \bullet \sigma)(y) = 6$, and

$$@(\sigma^*(t)) = 3 \cdot (6 + 1) = 21 = (@ \bullet \sigma)t.$$

The term-substitution lemma says that the square

$$\begin{array}{ccc} T_\Sigma X & \xrightarrow{\sigma^*} & T_\Sigma Y \\ \llbracket - \rrbracket_{\beta \bullet \sigma}^A \downarrow & & \downarrow \llbracket - \rrbracket_\beta^A \\ A & \xrightarrow{1_A} & A \end{array}$$

commutes. Equivalently, $\llbracket \sigma^*(t) \rrbracket_\beta^A = \llbracket t \rrbracket_{\beta \bullet \sigma}^A$.

It follows from naturality of μ and the algebra identity represented by the two adjacent commutative squares:

$$\begin{array}{ccccccc}
T_\Sigma X & \xrightarrow{T_\Sigma \sigma} & T_\Sigma^2 Y & \xrightarrow{T_\Sigma^2 \beta} & T_\Sigma^2 A & \xrightarrow{T_\Sigma a} & T_\Sigma A \\
& & \mu_Y \downarrow & & \mu_A \downarrow & & \downarrow a \\
& & T_\Sigma Y & \xrightarrow{T_\Sigma \beta} & T_\Sigma A & \xrightarrow{a} & A.
\end{array}$$

For the single-variable substitution $[x := s]$, the induced assignment is $\alpha \bullet [x := s] = \alpha[x \mapsto \llbracket s \rrbracket_\alpha^A]$. Consequently,

$$\boxed{\llbracket t[x := s] \rrbracket_\alpha^A = \llbracket t \rrbracket_{\alpha[x \mapsto \llbracket s \rrbracket_\alpha^A]}^A}$$

For $c \in A$, let $@_c^x$ be the assignment obtained from $@$ by sending x to c and leaving every other value fixed. For a term s , write $|_s^x t$ for $t[x := s]$. The same substitution law is then the type-revealing calculational identity

$$\boxed{(@|_s^x) t = @(|_s^x t).}$$

The commutative square records the types and compositional structure of the law; the $@$ -identity performs the corresponding calculation. Syntactic substitution is Kleisli composition, and its compatibility with evaluation is an Eilenberg–Moore algebra law. A concrete arithmetic comparison of the textbook, categorical, and $@$ -forms appears below.

7 Formulas and satisfaction

Let $\text{Form}_\mathcal{L}(X)$ be the set of α -equivalence classes of $\mathcal{L}$ -formulas whose free variables belong to X . Two formulas are α -equivalent when they differ only by capture-avoiding renaming of bound variables. A Kleisli substitution $\sigma : X \rightarrow T_\Sigma Y$ induces capture-avoiding substitution $(-)[\sigma] : \text{Form}_\mathcal{L}(X) \rightarrow \text{Form}_\mathcal{L}(Y)$, which is well-defined on these classes.

Put $Q(X) = \text{Form}_\mathcal{L}(X)$ and $Q(\sigma) = (-)[\sigma]$. On atomic formulas, $Q(\sigma)$ substitutes into their constituent terms. It commutes with the propositional connectives. At a quantifier, bound variables are renamed when required to prevent capture. The identity law is the commutativity of

$$\begin{array}{ccc}
& Q(X) & \\
Q(\eta_X) \swarrow & & \searrow 1_{Q(X)} \\
Q(X) & \xrightarrow{1_{Q(X)}} & Q(X),
\end{array}$$

and, for $\sigma : X \rightarrow T_\Sigma Y$ and $\tau : Y \rightarrow T_\Sigma Z$, the composition law is the commutativity of

$$\begin{array}{ccc}
Q(X) & \xrightarrow{Q(\sigma)} & Q(Y) \\
& \searrow Q(\tau \star \sigma) & \downarrow Q(\tau) \\
& & Q(Z).
\end{array}$$

Thus formulas carry an action of the Kleisli substitutions of the term monad: $Q : \mathbf{Kl}(T_\Sigma) \rightarrow \mathbf{Set}$ is a functor. Bound variables add the capture-avoidance requirement to this action on formulas; its composition laws remain those displayed above.

Satisfaction is defined in the usual way. Its atomic clauses are

$$\begin{aligned} \mathcal{A}, \alpha \models t = u &\iff \llbracket t \rrbracket_\alpha^A = \llbracket u \rrbracket_\alpha^A, \\ \mathcal{A}, \alpha \models R(t_1, \dots, t_n) &\iff (\llbracket t_1 \rrbracket_\alpha^A, \dots, \llbracket t_n \rrbracket_\alpha^A) \in R^A. \end{aligned}$$

For calculations, define the postfix notation $\varphi @ \iff \mathcal{A}, @ \models \varphi$. The atomic clauses become

$$(t = u) @ \iff @ t = @ u, \quad R(t_1, \dots, t_n) @ \iff (@ t_1, \dots, @ t_n) \in R^A.$$

When the structure must be shown, write $\varphi @_{\mathcal{A}}$.

The clauses for the propositional connectives use the corresponding operations in the metalanguage. Let $\alpha : X \rightarrow A$. For $b \in A$, write $\alpha[x \mapsto b] : X \cup \{x\} \rightarrow A$ for the assignment that sends x to b and agrees with α elsewhere; when $x \in X$, this overwrites its old value. If the free variables of φ lie in $X \cup \{x\}$, the quantifier clauses are

$$\begin{aligned} \mathcal{A}, \alpha \models (\forall x)\varphi &\iff \text{for every } b \in A, \mathcal{A}, \alpha[x \mapsto b] \models \varphi, \\ \mathcal{A}, \alpha \models (\exists x)\varphi &\iff \text{for some } b \in A, \mathcal{A}, \alpha[x \mapsto b] \models \varphi. \end{aligned}$$

Using $@|_b^x : X \cup \{x\} \rightarrow A$ for the assignment that sends x to b and otherwise agrees with $@$, the same clauses are

$$\begin{aligned} ((\forall x)\varphi) @ &\iff \text{for every } b \in A, \varphi (@|_b^x), \\ ((\exists x)\varphi) @ &\iff \text{for some } b \in A, \varphi (@|_b^x). \end{aligned}$$

Regard satisfaction as its characteristic map $S_X^A : \text{Form}_{\mathcal{L}}(X) \times A^X \rightarrow \mathbf{2}$, with value $\top$ exactly on satisfied pairs. For an assignment $\alpha : X \rightarrow A$, write $S_\alpha^A := S_X^A(-, \alpha)$. For $\sigma : X \rightarrow T_\Sigma Y$ and $\beta : Y \rightarrow A$, the formula-substitution lemma is the commutativity of

$$\begin{array}{ccc} \text{Form}_{\mathcal{L}}(X) & \xrightarrow{(-)[\sigma]} & \text{Form}_{\mathcal{L}}(Y) \\ S_{\beta \bullet \sigma}^A \downarrow & & \downarrow S_\beta^A \\ \mathbf{2} & \xrightarrow{1_2} & \mathbf{2}. \end{array}$$

Elementwise, the square says

$$\boxed{\mathcal{A}, \beta \models \varphi[\sigma] \iff \mathcal{A}, \beta \bullet \sigma \models \varphi}$$

provided substitution is capture-avoiding. Its atomic cases are consequences of the algebra law for term evaluation; its quantifier cases follow by induction after the required renaming of bound variables.

For a single-variable substitution, write $\varphi|_s^x$ for the capture-avoiding formula $\varphi[x := s]$. With the assignment-update notation introduced above, the same lemma is the calculational identity

$$\boxed{(\varphi|_s^x) @ \iff \varphi (@|_{@s}^x).$$

Example: avoiding capture. In arithmetic with $<$, let φ be $(\exists y) x < y$, with $\text{FV}(\varphi) = \{x\}$, and substitute the variable y for x . Capture-avoiding substitution yields $\varphi[x := y] = (\exists z) y < z$, with z fresh. Textual replacement without renaming the bound variable would yield $(\exists y) y < y$, changing the meaning. If $@(y) = 3$ in $\mathbb{N}$, then $(\varphi|_y^x)@ \iff \varphi(@|_y^x)$, and both sides are true; the formula produced by replacement without renaming is false.

Satisfaction also has the locality property. If assignments $\alpha, \alpha' : X \rightarrow A$ agree on every free variable of φ , then $\mathcal{A}, \alpha \models \varphi$ if and only if $\mathcal{A}, \alpha' \models \varphi$.

In textbook usage, the notation $\varphi(x_1, \dots, x_n)$, when introduced by saying that the free variables of φ are *among* $x_1, \dots, x_n$, means $\text{FV}(\varphi) \subseteq \{x_1, \dots, x_n\}$.

Some, all, or none of the displayed variables may actually occur freely.

8 The at-sign notation as a pairing calculus

The assignment, term-evaluation, and satisfaction maps defined above have two parallel presentations. Category-theoretic notation records their types and composition; the @-notation is used for their calculations. Prefix or postfix position identifies the typed pairing in use.

In this note, a **pairing** means simply a map $P \times Q \rightarrow K$ with two ordered inputs. The two pairings are

$$E_X^A : A^X \times T_\Sigma X \longrightarrow A, \quad S_X^A : \text{Form}_\mathcal{L}(X) \times A^X \longrightarrow \mathbf{2}.$$

Here A^X is the set of assignments $X \rightarrow A$, $E_X^A(\alpha, t) = \llbracket t \rrbracket_\alpha^A$, and, for $\mathbf{2} = \{\perp, \top\}$, $S_X^A(\varphi, \alpha) = \top$ exactly when $\mathcal{A}, \alpha \models \varphi$. Thus S_X^A is the characteristic function of the ordinary satisfaction relation.

The prefix and postfix notation already used above abbreviates these pairings: $@t = E_X^A(@, t)$, while $\varphi@$ holds exactly when $S_X^A(\varphi, @) = \top$.

The placement of @ records the assignment's argument position and therefore identifies the pairing being used. Prefix @ invokes E_X^A and returns an element of A ; postfix @ invokes S_X^A by abbreviating the assertion $S_X^A(\varphi, @) = \top$.

Example: the two result types. In the arithmetic structure on $\mathbb{N}$, let $@(x) = 2$ and $@(y) = 3$. Then $@(x \cdot (y + 1)) = 8 \in \mathbb{N}$, whereas $(x \cdot (y + 1) = 8)@$ is a true proposition.

The update and substitution symbols introduced above make substitution appear to pass through an evaluation pairing, from the syntactic argument to the assignment. For an arbitrary Kleisli substitution $\sigma : X \rightarrow T_\Sigma Y$, write $P_a(\sigma) : A^Y \rightarrow A^X$ for the assignment map $\beta \mapsto \beta \bullet \sigma$. The compatibility of term evaluation with substitution is the commutativity of the typed square

$$\begin{array}{ccc} A^Y \times T_\Sigma X & \xrightarrow{1_{A^Y} \times \sigma^*} & A^Y \times T_\Sigma Y \\ \downarrow P_a(\sigma) \times 1_{T_\Sigma X} & & \downarrow E_Y^A \\ A^X \times T_\Sigma X & \xrightarrow{E_X^A} & A. \end{array}$$

The compatibility of satisfaction with substitution is the commutativity of

$$\begin{array}{ccc} \text{Form}_\mathcal{L}(X) \times A^Y & \xrightarrow{(-)[\sigma] \times 1_{A^Y}} & \text{Form}_\mathcal{L}(Y) \times A^Y \\ \downarrow 1_{Q(X)} \times P_a(\sigma) & & \downarrow S_Y^A \\ \text{Form}_\mathcal{L}(X) \times A^X & \xrightarrow{S_X^A} & \mathbf{2}. \end{array}$$

In the @-calculus, for $@ : Y \rightarrow A$, the same two laws are

$$(@ \bullet \sigma)t = @(\sigma^*(t)), \quad (\varphi[\sigma]) @ \iff \varphi (@ \bullet \sigma).$$

The lower-right object in the satisfaction square is **2**, so its @-form is a biconditional. The diagrams and the @-display express the same compatibility of substitution with the evaluation pairings: the diagrams expose every type, while the @-forms perform the corresponding calculations.

8.1 One calculation in three notations

Work in $\mathbb{N}$ with $X = \{x, y\}$, $t = x \cdot (y + 1)$, $s = y + 1$, $\alpha(x) = 2$, and $\alpha(y) = 3$. Put $\sigma = [x := s]$. Since $\alpha \bullet \sigma = \alpha[x \mapsto 4]$, the same calculation has three forms:

- In traditional logical notation, $\llbracket t[x := s] \rrbracket_\alpha^\mathbb{N} = \llbracket t \rrbracket_{\alpha[x \mapsto 4]}^\mathbb{N} = 16$.
- In the @-calculus, the same equality is $(@|_{@s}^x)t = @(|_s^x t) = 16$, where $@ = \alpha$.
- Categorically, the substitution square says $E_X^\mathbb{N}(\alpha, \sigma^*(t)) = E_X^\mathbb{N}(\alpha \bullet \sigma, t)$: σ^* changes the syntax, while $\alpha \bullet \sigma$ changes the assignment.

Now take $\varphi = (\exists z) x < z$, with $\text{FV}(\varphi) = \{x\}$. In traditional notation, $\mathbb{N}, \alpha \models \varphi[x := s]$ if and only if $\mathbb{N}, \alpha \bullet \sigma \models \varphi$, where $\alpha \bullet \sigma = \alpha[x \mapsto 4]$. The categorical line is $S_X^\mathbb{N}(\varphi[\sigma], \alpha) = S_X^\mathbb{N}(\varphi, \alpha \bullet \sigma) = \top$, and the pairing line is $(\varphi|_s^x) @ \iff \varphi (@|_{@s}^x)$. The categorical line identifies the common substitution mechanism, while the @-line performs the corresponding calculation. Both remain alongside the textbook notation.

9 Occurrence-indexed substitution

Term and formula substitution construct a new syntactic expression. A second operation records where each occurrence of a formal argument goes during that construction. Došen–Petric use finite ordinals for this bookkeeping [1, §2.5, pp. 45–46]. Their operation is especially informative when an outer pattern repeats an argument or when an inserted pattern repeats one of its own arguments. Appendix A.5 applies the same finite occurrence construction to arithmetic term holes and proves its compatibility with Feferman’s substitution on codes.

The letters μ and ν_i denote occurrence maps in the original Došen–Petric notation. This note uses μ_X for monad multiplication and reserves ν_A for the naming map introduced later. Accordingly, write o_M and o_i for the occurrence maps in this section.

9.1 Finite occurrence blocks

For $n \in \mathbb{N}$, let $\underline{n} = \{1, \dots, n\}$, with $\underline{0} = \emptyset$. Given a finite list $\vec{a} = (a_1, \dots, a_m)$, define its tagged block sum by

$$\bigoplus_{r=1}^m \underline{a_r} = \{(r, t) : r \in \underline{m}, t \in \underline{a_r}\}.$$

This tagged disjoint union is the coproduct in **Set** of the finite blocks $\underline{a_r}$. The tag r records the block and t records the position inside that block. Put $|\vec{a}| = \sum_{r=1}^m \underline{a_r}$. The block encoding

$$\theta_{\vec{a}} : \bigoplus_{r=1}^m \underline{a_r} \longrightarrow |\vec{a}|, \quad \theta_{\vec{a}}(r, t) = \sum_{q < r} a_q + t$$

is a bijection. For $x \in |\vec{a}|$, there is a unique block r satisfying $\sum_{q < r} a_q < x \leq \sum_{q \leq r} a_q$; its inverse is

$$\theta_{\vec{a}}^{-1}(x) = \left(r, x - \sum_{q < r} a_q \right).$$

For example, when $\vec{a} = (2, 1, 3)$, the successive pairs $(1, 1), (1, 2), (2, 1), (3, 1), (3, 2), (3, 3)$ are encoded by 1, 2, 3, 4, 5, 6. Finite-ordinal treatments call $\theta_{\vec{a}}$ *flattening*. Its source is a tagged sum of finite sets. The term-monad map μ_X has source $T_\Sigma(T_\Sigma X)$ and removes a syntactic term layer. The two types keep the operations distinct.

The common zero-based convention $[0] = \emptyset$ and $[n] = \{0, \dots, n-1\}$ for $n \geq 1$ yields the same construction. The order isomorphism $\rho_n : \underline{n} \rightarrow [n]$, $\rho_n(t) = t-1$, simply subtracts one from each coordinate. In particular, $\rho_{|\vec{a}|}(\theta_{\vec{a}}(r, t)) = \sum_{q < r} a_q + (t-1)$, which is the zero-based block coordinate of the local position $t-1 \in [a_r]$. Mac Lane uses this zero-based convention, writing n for the finite ordinal denoted here by $[n]$ [2, §VII.5].

9.2 Occurrence profiles and fibrewise substitution

A **formula pattern** on a set P of placeholders is an expression built from the members of P , the truth constants $\top, \perp$, and propositional connectives. The truth constants are taken as primitive nullary logical connectives or as fixed closed formula abbreviations; in either case they determine elements of $\text{Form}_{\mathcal{L}}(X)$ for every variable set X . Write $\text{Pat}(P)$ for the set of such patterns. Replacing its placeholders by patterns or formulas while preserving the truth constants and connectives produces another pattern or a formula. If $P = \{p_1, \dots, p_k\}$, the pattern's **formal arity** is k . Its **occurrence slots** are the individual appearances of those placeholders, counted with repetitions and read from left to right.

Let $M \in \text{Pat}(P)$ occupy m occurrence slots. Its occurrence profile is the function

$$o_M : \underline{m} \longrightarrow \underline{k}, \quad o_M(r) = i \iff \text{the } r\text{-th occurrence is occupied by } p_i.$$

For comparison with the original Došen–Petrić notation, let $\text{sk}_P : \text{Pat}(P) \rightarrow \text{Pat}(\{\square\})$ be the erasure map defined recursively by replacing every formal placeholder in P by the single placeholder $\square$, while fixing truth constants and preserving connectives. When the placeholder set is clear, write sk . The decorated skeleton $\text{sk}_P(M)^{o_M}$ consists of the one-placeholder skeleton together with the external argument-indexing map o_M . Thus M and $\text{sk}_P(M)^{o_M}$ encode the same pattern data in two forms.

For each $i \in \underline{k}$, let $Q_i = \{q_{i,1}, \dots, q_{i,\ell_i}\}$, with all these placeholders pairwise distinct, and let $N_i \in \text{Pat}(Q_i)$ have n_i occurrence slots. Its occurrence profile is a function $o_i : \underline{n_i} \rightarrow \underline{\ell_i}$. Put $Q_{\text{occ}} = \coprod_{i=1}^k Q_i$, identified with the tagged set of placeholders $\{(i, q_{i,s}) : i \in \underline{k}, s \in \underline{\ell_i}\}$.

Define $\Phi_0 : P \rightarrow \text{Pat}(Q_{\text{occ}})$ by $\Phi_0(p_i) = N_i$, viewing N_i in $\text{Pat}(Q_{\text{occ}})$ through the inclusion $Q_i \rightarrow Q_{\text{occ}}$. Structural recursion yields a unique map $(-)[\Phi_0] : \text{Pat}(P) \rightarrow \text{Pat}(Q_{\text{occ}})$ that sends each p_i to N_i , fixes $\top$ and $\perp$, and preserves every propositional connective. Write $M[\Phi_0]$ for the resulting substituted pattern. While forming $M[\Phi_0]$, the r -th occurrence of M is replaced by a copy of $N_{o_M(r)}$.

Write $\vec{n} = (n_1, \dots, n_k)$, $\vec{\ell} = (\ell_1, \dots, \ell_k)$, and $\vec{o} = (o_1, \dots, o_k)$.

The occurrences in the substituted pattern form the tagged source

$$D_{M, \vec{n}} = \bigoplus_{r=1}^m \underline{n_{o_M(r)}} = \{(r, t) : r \in \underline{m}, t \in \underline{n_{o_M(r)}}\}.$$

The pair (r, t) means: first choose the r -th occurrence of M , then choose the t -th occurrence inside the inserted copy of $N_{o_M(r)}$. The formal arguments of all the inserted patterns form the tagged target

$$C_{\vec{\ell}} = \bigoplus_{i=1}^k \underline{\ell_i} = \{(i, s) : i \in \underline{k}, s \in \underline{\ell_i}\}.$$

The fibrewise occurrence-substitution function is

$$\gamma_{M,\vec{\sigma}} : D_{M,\vec{n}} \longrightarrow C_{\vec{\ell}}, \quad \gamma_{M,\vec{\sigma}}(r, t) = (o_M(r), o_{o_M(r)}(t)).$$

This formula is well-defined. Membership $t \in \underline{n_{o_M(r)}}$ and the declared type of $o_{o_M(r)}$ imply $o_{o_M(r)}(t) \in \underline{\ell_{o_M(r)}}$; hence the resulting pair lies in the $o_M(r)$ -th target fibre.

Let $\iota_r : \underline{n_{o_M(r)}} \rightarrow D_{M,\vec{n}}$ and $j_i : \underline{\ell_i} \rightarrow C_{\vec{\ell}}$ be the functions $\iota_r(t) = (r, t)$ and $j_i(s) = (i, s)$. They are the coproduct injections of the tagged block sums. For each $r \in \underline{m}$, the square

$$\begin{array}{ccc} \underline{n_{o_M(r)}} & \xrightarrow{\iota_r} & D_{M,\vec{n}} \\ \downarrow o_{o_M(r)} & & \downarrow \gamma_{M,\vec{\sigma}} \\ \underline{\ell_{o_M(r)}} & \xrightarrow{j_{o_M(r)}} & C_{\vec{\ell}} \end{array}$$

commutes. Taken together for every r , these squares determine $\gamma_{M,\vec{\sigma}}$ uniquely, because every element of $D_{M,\vec{n}}$ lies in exactly one injected fibre. This is the coproduct unique-extension property for the family of maps $j_{o_M(r)} \circ o_{o_M(r)}$.

Projection to the block tags yields another typed description. Define $p_D(r, t) = r$ and $p_C(i, s) = i$. The following square commutes:

$$\begin{array}{ccc} D_{M,\vec{n}} & \xrightarrow{\gamma_{M,\vec{\sigma}}} & C_{\vec{\ell}} \\ \downarrow p_D & & \downarrow p_C \\ \underline{m} & \xrightarrow{o_M} & \underline{k} \end{array}$$

On the fibre above r , the map is exactly $o_{o_M(r)}$, with its value placed in the fibre above $o_M(r)$.

9.3 The ordinal-coded operation

Define cumulative source and target sizes by

$$\pi(0) = 0, \quad \pi(r) = \sum_{q=1}^r n_{o_M(q)}, \quad \lambda(0) = 0, \quad \lambda(i) = \sum_{q=1}^i \ell_q.$$

Thus $D_{M,\vec{n}}$ has $\pi(m)$ elements and $C_{\vec{\ell}}$ has $\lambda(k)$ elements. Their block encodings are

$$\begin{aligned} \theta_D : D_{M,\vec{n}} &\longrightarrow \underline{\pi(m)}, & \theta_D(r, t) &= \pi(r-1) + t, \\ \theta_C : C_{\vec{\ell}} &\longrightarrow \underline{\lambda(k)}, & \theta_C(i, s) &= \lambda(i-1) + s. \end{aligned}$$

The Došen–Petrić ordinal-coded substitution is the transported function

$$\delta_{M,\vec{\sigma}} = \theta_C \circ \gamma_{M,\vec{\sigma}} \circ \theta_D^{-1} : \underline{\pi(m)} \longrightarrow \underline{\lambda(k)}.$$

Its relation to the fibrewise map is the commutative square

$$\begin{array}{ccc} D_{M,\vec{n}} & \xrightarrow{\gamma_{M,\vec{\sigma}}} & C_{\vec{\ell}} \\ \downarrow \theta_D & & \downarrow \theta_C \\ \underline{\pi(m)} & \xrightarrow{\delta_{M,\vec{\sigma}}} & \underline{\lambda(k)}. \end{array}$$

The coordinate formula uses the block locator $b : \pi(\underline{m}) \rightarrow \underline{m}$, defined by

$$b(x) = \min\{r \in \underline{m} : x \leq \pi(r)\}.$$

For every $x \in \pi(\underline{m})$, the displayed set is nonempty because $x \leq \pi(m)$; its least member is therefore defined and unique. The local position of x inside that source block is $x - \pi(b(x) - 1)$. Consequently,

$$\delta_{M,\vec{\sigma}}(x) = \lambda(o_M(b(x)) - 1) + o_{o_M(b(x))}(x - \pi(b(x) - 1)).$$

To verify the formula, start with x . The inverse block encoding satisfies $\theta_D^{-1}(x) = (b(x), x - \pi(b(x) - 1))$. Applying $\gamma_{M,\vec{\sigma}}$ selects the profile $o_{o_M(b(x))}$. Applying θ_C adds the target offset $\lambda(o_M(b(x)) - 1)$. This is exactly the displayed value of $\delta_{M,\vec{\sigma}}(x)$.

Occurrence-profile theorem. Under the block encodings θ_D and θ_C , the occurrence profile of the substituted pattern $M[\Phi_0]$ is $\delta_{M,\vec{\sigma}}$. Consequently, the labelled-pattern construction transports the Došen–Petric generalized composite to

$$\text{sk}_P(M)^{o_M}(\text{sk}_{Q_1}(N_1)^{o_1}, \dots, \text{sk}_{Q_k}(N_k)^{o_k}) := \text{sk}_{Q_{\text{occ}}}(M[\Phi_0])^{\delta_{M,\vec{\sigma}}},$$

where a superscript records the external argument-indexing map attached to the displayed skeleton. Renaming the skeletons $\text{sk}_P(M), \text{sk}_{Q_1}(N_1), \dots, \text{sk}_{Q_k}(N_k)$ as $M, N_1, \dots, N_k$, as in the original notation, this is $M^\mu(N_1^{\nu_1}, \dots, N_k^{\nu_k})$, with $\mu = o_M$, $\nu_i = o_i$, and $\mu(\nu_1, \dots, \nu_k) = \delta_{M,\vec{\sigma}}$.

Indeed, every occurrence of $M[\Phi_0]$ has unique fibre coordinates $(r, t) \in D_{M,\vec{n}}$: r selects an occurrence of M , and t selects an occurrence in the inserted copy of $N_{o_M(r)}$. The placeholder at that occurrence is the tagged element $(o_M(r), q_{o_M(r)}, o_{o_M(r)}(t)) \in Q_{\text{occ}}$. Under the canonical indexing of Q_{occ} by $C_{\vec{\ell}}$, its coordinate is $\gamma_{M,\vec{\sigma}}(r, t)$. Transporting source and target coordinates through θ_D and θ_C yields $\theta_C \circ \gamma_{M,\vec{\sigma}} \circ \theta_D^{-1} = \delta_{M,\vec{\sigma}}$, as required.

For maps $f_i : \underline{a_i} \rightarrow \underline{b_i}$, put $\vec{a} = (a_1, \dots, a_k)$ and $\vec{b} = (b_1, \dots, b_k)$. Their flattened ordinal sum is

$$f_1 \oplus \dots \oplus f_k = \theta_{\vec{b}} \circ \left(\bigoplus_{i=1}^k f_i \right) \circ \theta_{\vec{a}}^{-1}, \quad \left(\bigoplus_{i=1}^k f_i \right)(i, t) = (i, f_i(t)).$$

When $m = k$ and $o_M = 1_{\underline{k}}$, each outer slot uses the inserted pattern with the same index. The fibrewise map is $\bigoplus_{i=1}^k o_i$, and its ordinal-coded form is $o_1 \oplus \dots \oplus o_k$.

9.4 A complete occurrence calculation

Take the formula patterns

$$M(p, q) = q \wedge (p \vee q), \quad N_1(a, b) = a \wedge b, \quad N_2(c, d) = d \wedge (c \vee d).$$

Reading occurrences from left to right yields $o_M = (2, 1, 2)$, $o_1 = (1, 2)$, and $o_2 = (2, 1, 2)$.

Here $m = 3$, $k = 2$, $(n_1, n_2) = (2, 3)$, and $(\ell_1, \ell_2) = (2, 2)$. After substitution,

$$M(N_1, N_2) = (d \wedge (c \vee d)) \wedge ((a \wedge b) \vee (d \wedge (c \vee d))).$$

Its source fibres have sizes 3, 2, 3, while its target fibres list the formal arguments as (a, b) and (c, d) . The successive source occurrences and their encoded target positions are

source position x	1	2	3	4	5	6	7	8
occurring letter	d	c	d	a	b	d	c	d
$\delta_{M,\vec{\sigma}}(x)$	4	3	4	1	2	4	3	4

Thus the generalized composite in the labelled-pattern encoding of the Došen–Petric notation is

$$\text{sk}_P(M)^{(2,1,2)}(\text{sk}_{Q_1}(N_1)^{(1,2)}, \text{sk}_{Q_2}(N_2)^{(2,1,2)}) = \text{sk}_{Q_{\text{occ}}}(M(N_1, N_2))^{(4,3,4,1,2,4,3,4)}.$$

For instance, $\theta_D^{-1}(2) = (1, 2)$: source position 2 is the second occurrence inside the copy of N_2 inserted at the first outer slot. Then $\gamma_{M,\bar{\sigma}}(1, 2) = (2, o_2(2)) = (2, 1)$, and $\theta_C(2, 1) = 3$. Hence $\delta_{M,\bar{\sigma}}(2) = 3$, as the table records.

Three substitution operations now have explicit types. The map $\sigma^* : T_\Sigma X \rightarrow T_\Sigma Y$ substitutes terms for term variables. The map $(-)[\sigma] : \text{Form}_\mathcal{L}(X) \rightarrow \text{Form}_\mathcal{L}(Y)$ performs capture-avoiding term substitution in formulas. The map $(-)[\Phi_0] : \text{Pat}(P) \rightarrow \text{Pat}(Q_{\text{occ}})$ substitutes formula patterns for formal formula arguments. The maps $\gamma_{M,\bar{\sigma}}$ and $\delta_{M,\bar{\sigma}}$ record the intermediate pattern-level provenance of placeholder occurrences under the third operation.

9.5 The @-calculus for the associated formula substitution

The @-calculus evaluates the formulas produced by the pattern substitution whose occurrence provenance is recorded by $\gamma_{M,\bar{\sigma}}$ or $\delta_{M,\bar{\sigma}}$. For every $i \in \underline{k}$ and $s \in \underline{\ell}_i$, choose an actual $\mathcal{L}$ -formula $\psi_{i,s}$ whose free variables belong to X , and define $\Psi_i : Q_i \rightarrow \text{Form}_\mathcal{L}(X)$ by $\Psi_i(q_{i,s}) = \psi_{i,s}$. Recursively replace every $q_{i,s}$ in N_i by $\psi_{i,s}$, fixing $\top, \perp$, and preserving each connective, and write the resulting formula as $\hat{N}_i = N_i[\Psi_i]$. The outer formula-pattern substitution is the function

$$\Phi : P \longrightarrow \text{Form}_\mathcal{L}(X), \quad \Phi(p_i) = \hat{N}_i.$$

Structural recursion yields a unique truth-constant-and-connective-preserving extension $(-)[\Phi] : \text{Pat}(P) \rightarrow \text{Form}_\mathcal{L}(X)$: it sends p_i to $\Phi(p_i)$ and sends each connective to the same connective applied to the recursively substituted arguments. Write $M[\Phi]$ for the image of M under this extension.

The maps Ψ_i combine uniquely to $\Psi : Q_{\text{occ}} \rightarrow \text{Form}_\mathcal{L}(X)$. Structural recursion yields its unique extension $(-)[\Psi] : \text{Pat}(Q_{\text{occ}}) \rightarrow \text{Form}_\mathcal{L}(X)$. Associativity of pattern substitution is the map identity

$$(-)[\Psi] \circ (-)[\Phi_0] = (-)[\Phi],$$

and hence, at M , the equation

$$(M[\Phi_0])[\Psi] = M[\Phi].$$

Both sides are obtained by the unique extension that sends p_i to $N_i[\Psi_i]$, fixes $\top, \perp$, and preserves every connective. The left side is therefore the formula obtained by instantiating the pattern underlying the Došen–Petric generalized composite.

For an $\mathcal{L}$ -structure $\mathcal{A}$ and an assignment $@ : X \rightarrow A$, define the induced truth assignment

$$@ \bullet \Phi : P \longrightarrow \mathbf{2}, \quad (@ \bullet \Phi)(p_i) = S_X^A(\Phi(p_i), @).$$

Write $@_\Phi := @ \bullet \Phi$ for this induced truth assignment. This use of $\bullet$ is the truth-valued analogue of the induced assignment $\beta \bullet \sigma$ for term substitution. The truth evaluation pairing has type

$$S_P^2 : \text{Pat}(P) \times \mathbf{2}^P \longrightarrow \mathbf{2}.$$

For $v : P \rightarrow \mathbf{2}$, it is defined recursively by $S_P^2(p_i, v) = v(p_i)$, $S_P^2(\top, v) = \top$, and $S_P^2(\perp, v) = \perp$. For example, $S_P^2(\neg U, v) = \neg S_P^2(U, v)$ and $S_P^2(U \wedge V, v) = S_P^2(U, v) \wedge S_P^2(V, v)$; every further connective uses its corresponding truth function.

For a truth assignment $@_P : P \rightarrow \mathbf{2}$, let $S_{@_P}^2 = S_P^2(-, @_P) : \text{Pat}(P) \rightarrow \mathbf{2}$, and write $M @_P \rightleftharpoons S_P^2(M, @_P) = \top$. Substitution of the formulas $\Phi(p_i)$ for the formal arguments is compatible with the two satisfaction pairings, as expressed by the following commutative square:

$$\begin{array}{ccc} \text{Pat}(P) & \xrightarrow{(-)[\Phi]} & \text{Form}_{\mathcal{L}}(X) \\ S_{@_\Phi}^2 \downarrow & & \downarrow S_{@_\Phi}^A \\ \mathbf{2} & \xrightarrow{1_2} & \mathbf{2} \end{array}$$

In the $@$ -calculus, this is the sliding law

$$\boxed{(M[\Phi]) @ \iff M @_\Phi.}$$

The law follows by structural induction on M . At the placeholder p_i , it is the defining equation for $@_\Phi = @ \bullet \Phi$. At $\top$ and $\perp$, it is their respective truth clause. Each connective step follows from the corresponding truth function.

Combining this sliding law with pattern-substitution associativity applies the $@$ -calculus directly to the instantiated Došen–Petrić composite:

$$\boxed{((M[\Phi_0])[\Psi]) @ \iff M @_\Phi.}$$

The occurrence maps have a direct $@$ -labelled form. Give every formal argument of every inserted pattern its truth value under the current assignment by defining

$$e_{@} : C_{\vec{\ell}} \longrightarrow \mathbf{2}, \quad e_{@}(i, s) = S_X^A(\psi_{i,s}, @), \quad d_{@} : D_{M, \vec{n}} \longrightarrow \mathbf{2}, \quad d_{@} = e_{@} \circ \gamma_{M, \vec{\sigma}}.$$

In the postfix $@$ -notation, these definitions say

$$\begin{aligned} e_{@}(i, s) = \top & \iff \psi_{i,s} @, \\ d_{@}(r, t) = \top & \iff \psi_{o_M(r), o_{o_M(r)}(t)} @. \end{aligned}$$

Thus $d_{@}$ labels each pattern-level placeholder occurrence in the substituted pattern by pulling back the truth label of the formal argument from which it came. Occurrences internal to an actual formula $\psi_{i,s}$ are represented here by the single truth value of that formula. In fibre coordinates this is the commutative triangle

$$\begin{array}{ccc} D_{M, \vec{n}} & \xrightarrow{\gamma_{M, \vec{\sigma}}} & C_{\vec{\ell}} \\ & \searrow d_{@} & \downarrow e_{@} \\ & & \mathbf{2}. \end{array}$$

After the block encodings, put $\bar{e}_{@} = e_{@} \circ \theta_C^{-1}$ and $\bar{d}_{@} = d_{@} \circ \theta_D^{-1}$. The corresponding ordinal-coordinate triangle is

$$\begin{array}{ccc} \pi(m) & \xrightarrow{\delta_{M, \vec{\sigma}}} & \lambda(k) \\ & \searrow \bar{d}_{@} & \downarrow \bar{e}_{@} \\ & & \mathbf{2}. \end{array} \quad \bar{d}_{@} = \bar{e}_{@} \circ \delta_{M, \vec{\sigma}}.$$

These diagrams are the direct interface between the @-calculus and the occurrence operation: truth evaluation supplies the pattern-level formal-argument occurrence labels, and the occurrence maps transport those labels through substitution. Evaluating the connective tree of $M[\Phi]$ then returns its single truth value.

For the preceding example, replace a, b, c, d by actual formulas $\psi_a, \psi_b, \psi_c, \psi_d$. Thus $\hat{N}_1 = \psi_a \wedge \psi_b$, $\hat{N}_2 = \psi_d \wedge (\psi_c \vee \psi_d)$, $\Phi(p) = \hat{N}_1$, and $\Phi(q) = \hat{N}_2$. Suppose $\psi_a @$, $\psi_b @$, and $\psi_c @$ are true, while $\psi_d @$ is false. In the target order (a, b, c, d) , the truth labels and their pullback along the computed occurrence map are

$$\bar{e}_@ = (\top, \top, \top, \perp), \quad \bar{d}_@ = \bar{e}_@ \circ \delta_{M, \sigma} = (\perp, \top, \perp, \top, \top, \perp, \top, \perp).$$

The induced assignment to the outer arguments satisfies $@_\Phi(p) = \top$ and $@_\Phi(q) = \perp$. Hence

$$S_P^2(M, @_\Phi) = \perp \wedge (\top \vee \perp) = \perp,$$

and the sliding law says that $(M[\Phi]) @$ is false.

10 Expansion of a language by names for a structure

Let $\mathcal{A}$ be an $\mathcal{L}$ -structure with domain A . The expansion of $\mathcal{L}$ by names for elements of A , written here as $\mathcal{L}_A$, adds a new constant symbol $\bar{c}$ for every $c \in A$. The symbols $\bar{c}$ form a disjoint copy of A : the element c lies in the domain, while its name $\bar{c}$ is a formal constant in the expanded signature.

Let $\Sigma[A]$ be the expanded functional signature. A term over $\Sigma[A]$ with variables in X is equivalently an original Σ -term whose leaves may be either named elements of A or variables in X .

This description by leaves yields a natural bijection $\chi_X : T_{\Sigma[A]}X \xrightarrow{\cong} T_\Sigma(A + X)$.

Here $A + X$ is the disjoint union, or coproduct in **Set**, with canonical inclusions $\text{inl} : A \rightarrow A + X$ and $\text{inr} : X \rightarrow A + X$. Under this identification, the A -summand is held fixed by substitution, while the X -summand contains the substitutable variables.

This bijection is **natural in** X : for every $u : X \rightarrow Y$, the square

$$\begin{array}{ccc} T_{\Sigma[A]}X & \xrightarrow{\chi_X} & T_\Sigma(A + X) \\ T_{\Sigma[A]}u \downarrow & & \downarrow T_\Sigma(1_A + u) \\ T_{\Sigma[A]}Y & \xrightarrow{\chi_Y} & T_\Sigma(A + Y) \end{array}$$

commutes. The monad structure on $X \mapsto T_\Sigma(A + X)$ in this description is the expanded term-monad structure transported through χ .

The inclusion of the original signature into the expanded signature induces the monad morphism $j : T_\Sigma \Rightarrow T_{\Sigma[A]}$. Under the displayed identification, its component is typed by the commutative triangle

$$\begin{array}{ccc} T_\Sigma X & \xrightarrow{j_X} & T_{\Sigma[A]}X \\ & \searrow T_\Sigma(\text{inr}) & \downarrow \chi_X \\ & & T_\Sigma(A + X) \end{array}$$

Here $\text{inr} : X \rightarrow A + X$ includes the variable summand. Thus j regards every original term as an expanded-language term without inserting any new constants. Because j is a monad morphism, this inclusion preserves variables and syntactic substitution.

More generally, to give a $\Sigma[A]$ -algebra structure on a set B is to give:

- a T_Σ -algebra $b : T_\Sigma B \rightarrow B$; and
- a function $k : A \rightarrow B$ interpreting the newly added constants.

The full expanded algebra map $h_{b,k} : T_{\Sigma[A]}B \rightarrow B$ is the outer path in

$$T_{\Sigma[A]}B \xrightarrow{\chi_B} T_\Sigma(A+B) \xrightarrow{T_\Sigma[k, 1_B]} T_\Sigma B \xrightarrow{b} B.$$

$\curvearrowright$
 $h_{b,k}$

Here $[k, 1_B] : A + B \rightarrow B$ is the **copairing** of k and the identity of B : it sends the A -summand by k and the B -summand by 1_B . It is the unique function with those two restrictions.

The expanded algebra retains the original Σ -operations.

Their preservation is expressed by the following commutative restriction triangle:

$$\begin{array}{ccc} T_\Sigma B & \xrightarrow{j_B} & T_{\Sigma[A]}B \\ & \searrow b & \downarrow h_{b,k} \\ & & B. \end{array}$$

The distinguished expansion $\mathcal{A}^+$ of $\mathcal{A}$ takes $B = A$, retains every original relation R^A , and uses the pair $(a, 1_A)$: the original algebra map $a : T_\Sigma A \rightarrow A$ together with the identity interpretation of the new names. Denote the resulting specialization of the preceding path by $a^+ : T_{\Sigma[A]}A \rightarrow A$. In particular, $\bar{c}^{\mathcal{A}^+} = c$ for every $c \in A$.

Restricting the functional reduct of $\mathcal{A}^+$ along the monad morphism j recovers the functional reduct of $\mathcal{A}$: this is the specialization $B = A$, $b = a$, and $k = 1_A$ of the restriction triangle. Forgetting the newly named constants changes none of the original operations.

For ground terms, let $\rho_A : A + \emptyset \xrightarrow{\cong} A$ be the canonical bijection. Ground-term evaluation is the map

$$T_{\Sigma[A]}\emptyset \xrightarrow{\chi_\emptyset} T_\Sigma(A + \emptyset) \xrightarrow{T_\Sigma(\rho_A)} T_\Sigma A \xrightarrow{a} A.$$

$\curvearrowright$
 e_A

After the canonical identification $A + \emptyset \cong A$, this map is the original algebra map $a : T_\Sigma A \rightarrow A$.

The naming map $\nu_A : A \rightarrow T_{\Sigma[A]}\emptyset$, given by $c \mapsto \bar{c}$, is related to the original monad unit by the commutative square

$$\begin{array}{ccc} A & \xrightarrow{\nu_A} & T_{\Sigma[A]}\emptyset \\ \text{inl} \downarrow & & \downarrow \chi_\emptyset \\ A + \emptyset & \xrightarrow{\eta_{A+\emptyset}^{T_\Sigma}} & T_\Sigma(A + \emptyset). \end{array}$$

After the canonical identification $A + \emptyset \cong A$, the square says that ν_A corresponds to $\eta_A^{T_\Sigma} : A \rightarrow T_\Sigma A$. The algebra unit triangle then makes the following section triangle commute:

$$\begin{array}{ccc} A & \xrightarrow{\nu_A} & T_{\Sigma[A]} \emptyset \\ & \searrow 1_A & \downarrow e_A \\ & & A. \end{array}$$

Thus every element has a chosen ground-term name, and evaluating that name returns the element. A **ground term** is a term with no variables. The section triangle says that ν_A is a **section**, or right inverse, of ground-term evaluation. It need not be a two-sided inverse: many different ground terms can have the same value in $\mathcal{A}^+$.

Let $\epsilon_A : \emptyset \rightarrow A$ be the unique empty assignment, and write $@_{\emptyset, \mathcal{A}^+} := \epsilon_A$ when evaluation takes place in $\mathcal{A}^+$. In this two-subscript notation for empty assignments, the first subscript records the variable context and the second specifies the structure. In the pairing calculus, the section equation says that evaluating a name returns its value: $@_{\emptyset, \mathcal{A}^+} \bar{c} = c$, where $c \in A$.

11 Assignments as substitutions of names

The expansion $\mathcal{L}_A$ converts an assignment into a syntactic substitution by ground terms. Given $\alpha : X \rightarrow A$, define $\bar{\alpha} := \nu_A \circ \alpha : X \rightarrow T_{\Sigma[A]} \emptyset$.

This definition is encoded by the commutative triangle

$$\begin{array}{ccccc} X & \xrightarrow{\alpha} & A & \xrightarrow{\nu_A} & T_{\Sigma[A]} \emptyset. \\ & & \searrow \bar{\alpha} & & \end{array}$$

The Kleisli substitution $\bar{\alpha}$ sends each variable x to the constant $\overline{\alpha(x)}$. Regard every original term as an expanded-language term by j_X . The inclusion of languages also induces $\iota_X : \text{Form}_{\mathcal{L}}(X) \rightarrow \text{Form}_{\mathcal{L}_A}(X)$. Throughout this section, an original formula is regarded as an expanded-language formula by ι_X . To keep the calculations readable, write $t[\bar{\alpha}]$ for $\bar{\alpha}^*(j_X(t))$ and $\varphi[\bar{\alpha}]$ for $\iota_X(\varphi)[\bar{\alpha}]$.

Categorical form. For $t \in T_\Sigma X$ and $\varphi \in \text{Form}_{\mathcal{L}}(X)$, evaluation factors through the language inclusion and the Kleisli extension of $\bar{\alpha}$. The two factorizations are the outer paths in the following typed diagrams:

$$\begin{array}{ccccc} T_\Sigma X & \xrightarrow{j_X} & T_{\Sigma[A]} X & \xrightarrow{\bar{\alpha}^*} & T_{\Sigma[A]} \emptyset \\ & \searrow & & & \downarrow e_A \\ & & & & A \\ & \searrow E_X^A(\alpha, -) & & & \\ \text{Form}_{\mathcal{L}}(X) & \xrightarrow{\iota_X} & \text{Form}_{\mathcal{L}_A}(X) & \xrightarrow{(-)[\bar{\alpha}]} & \text{Form}_{\mathcal{L}_A}(\emptyset) \\ & \searrow & & & \downarrow S_{\emptyset}^{A^+}(-, \epsilon_A) \\ & & & & \mathbf{2} \\ & \searrow S_X^A(-, \alpha) & & & \end{array}$$

The first diagram applies ground-term evaluation to $t[\bar{\alpha}]$. The second evaluates the truth value of the sentence $\varphi[\bar{\alpha}]$; replacing every free variable by a name leaves the empty assignment as its only assignment argument.

At-sign calculation. Set $@ = \alpha$, write $\bar{@} := \nu_A \circ @$, and evaluate the resulting closed expressions using $@_{\emptyset, \mathcal{A}^+}$. The same two maps satisfy the calculation rules

$$\boxed{@_{\mathcal{A}} t = @_{\emptyset, \mathcal{A}^+} (t[\bar{@}]), \quad \varphi @_{\mathcal{A}} \iff \varphi[\bar{@}] @_{\emptyset, \mathcal{A}^+}.}$$

Thus the bar over $@$ changes an assignment into a substitution of names, while the prefix or postfix position of the assignment symbol marks term evaluation or satisfaction. The right-hand expressions are evaluated in the distinguished expanded structure $\mathcal{A}^+$.

Traditional notation. Ordinary evaluation brackets and satisfaction notation express the two rules as follows:

$$[[t]]_{\alpha}^{\mathcal{A}} = [[t[\bar{\alpha}]]]_{\epsilon_A}^{\mathcal{A}^+}, \quad \mathcal{A}, \alpha \models \varphi \iff \mathcal{A}^+, \epsilon_A \models \varphi[\bar{\alpha}].$$

Assignment modification is also represented syntactically. For $x \in X$ and $c \in A$, $\bar{\alpha}[x \mapsto \bar{c}] = \bar{\alpha}[x \mapsto \bar{c}]$. For the following equations, write $t[x := \bar{c}]$ for $j_X(t)[x := \bar{c}]$ and $\varphi[x := \bar{c}]$ for $\iota_X(\varphi)[x := \bar{c}]$, regarding the ground constant $\bar{c}$ as a term in the variable context X . The structures $\mathcal{A}$ and $\mathcal{A}^+$ have the same domain, so the same function $\alpha : X \rightarrow A$ is an assignment in both. The substitution lemmas yield

$$[[t]]_{\alpha[x \mapsto \bar{c}]}^{\mathcal{A}} = [[t[x := \bar{c}]]]_{\alpha}^{\mathcal{A}^+}, \\ \mathcal{A}, \alpha[x \mapsto \bar{c}] \models \varphi \iff \mathcal{A}^+, \alpha \models \varphi[x := \bar{c}].$$

In the $@$ -calculus, where the structure subscript distinguishes evaluation in $\mathcal{A}$ from evaluation in $\mathcal{A}^+$, these are

$$\boxed{(@|_c^x)_{\mathcal{A}} t = @_{\mathcal{A}^+} (|_{\bar{c}}^x t), \quad \varphi (@|_c^x)_{\mathcal{A}} \iff (\varphi|_{\bar{c}}^x) @_{\mathcal{A}^+}.}$$

The expansion therefore internalizes assignments as substitutions of constant names. It supplies a chosen ground-term name for every element, and the algebra map evaluates those terms back into the structure.

Example: replacing an assignment by names. Let $\mathbb{N}^+$ add a constant $\bar{n}$ for every natural number. Take $X = \{x, y, z\}$, $t = x \cdot (y + 1)$, $\varphi = (t = z)$, and an assignment $@ : X \rightarrow \mathbb{N}$ with $@(x) = 2$, $@(y) = 3$, and $@(z) = 8$. Categorically, $\bar{@} = \nu_{\mathbb{N}} \circ @$ is the ground-term substitution with $t[\bar{@}] = \bar{2} \cdot (\bar{3} + 1)$ and $\varphi[\bar{@}] = (\bar{2} \cdot (\bar{3} + 1) = \bar{8})$. The corresponding $@$ -calculation is

$$@_{\mathbb{N}} t = @_{\emptyset, \mathbb{N}^+} (\bar{2} \cdot (\bar{3} + 1)) = 2 \cdot (3 + 1) = 8, \quad \varphi @_{\mathbb{N}} \iff (\bar{2} \cdot (\bar{3} + 1) = \bar{8}) @_{\emptyset, \mathbb{N}^+}.$$

The ordinary textbook statement is $\mathbb{N}, @ \models x \cdot (y + 1) = z$ if and only if the sentence $\bar{2} \cdot (\bar{3} + 1) = \bar{8}$ is true in $\mathbb{N}^+$. The categorical map and the $@$ -calculation express the same replacement of an assignment by constant names.

12 Further perspective: functorial models and generalized points

This section may be skipped on a first reading. The functorial reformulation extends term algebras, assignment actions, and satisfaction from set-based models to models valued in other categories.

The action of substitutions on assignments already defines a functor. Fix the functional structure (A, a) . Assign to each variable set X its set A^X of assignments, and send a Kleisli substitution $\sigma : X \rightarrow T_{\Sigma} Y$ to the map $P_a(\sigma) : A^Y \rightarrow A^X$ defined earlier.

For $\sigma : X \rightarrow T_\Sigma Y$ and $\tau : Y \rightarrow T_\Sigma Z$, the composition and identity laws are typed by the following commutative triangles:

$$\begin{array}{ccc}
 A^Z & \xrightarrow{P_a(\tau)} & A^Y \\
 & \searrow P_a(\tau \star \sigma) & \downarrow P_a(\sigma) \\
 & & A^X.
 \end{array}
 \qquad
 \begin{array}{ccc}
 & A^X & \\
 P_a(\eta_X) \swarrow & & \searrow 1_{A^X} \\
 A^X & \xrightarrow{1_{A^X}} & A^X.
 \end{array}$$

These laws make $P_a : \mathbf{Kl}(T_\Sigma)^{\text{op}} \rightarrow \mathbf{Set}$ a contravariant functor, with $P_a(X) = A^X$. The algebra law therefore organizes assignments over all variable contexts at once.

For a finitary term monad, the assignment functor is the set-valued instance of Lawvere’s functorial semantics [3]. Let $\mathbb{L}_\Sigma$ be the full subcategory of $\mathbf{Kl}(T_\Sigma)^{\text{op}}$ on finite cardinals. Its arrows satisfy $\mathbb{L}_\Sigma(n, m) \cong (T_\Sigma n)^m$: an arrow $n \rightarrow m$ is an m -tuple of terms in n variables. This is the **Lawvere theory** of Σ . Finite products generalize Cartesian products by their universal mapping property. A model in a category $\mathcal{C}$ with finite products is a product-preserving functor $M : \mathbb{L}_\Sigma \rightarrow \mathcal{C}$; that is, it sends the abstract product diagrams to product diagrams. Natural transformations between models are homomorphisms. Taking $\mathcal{C} = \mathbf{Set}$ recovers the T_Σ -algebras used above; for an equational theory, one instead quotients the arrows by provable equality of terms.

Changing the target category produces further examples. The Lawvere theory of groups yields ordinary groups in **Set**, topological groups in **Top**, sheaves of groups in **Sh**(S), and group schemes over S in **Sch**/ S .

For a homomorphism $h : G \rightarrow H$, naturality at multiplication is the familiar commutative square

$$\begin{array}{ccc}
 G \times G & \xrightarrow{m_G} & G \\
 h \times h \downarrow & & \downarrow h \\
 H \times H & \xrightarrow{m_H} & H.
 \end{array}$$

Its commutativity is the equation $h(g_1 g_2) = h(g_1) h(g_2)$. A morphism of theories induces a reduct functor between model categories by precomposition, while a finite-product-preserving functor between target categories transports models by postcomposition.

12.1 Relations, geometric logic, and classifying toposes

A Lawvere theory describes operations and equations. Relation symbols and the operations of geometric logic—finite conjunction, arbitrary disjunction, and existential quantification—require a richer syntactic category. For a geometric theory $\mathbb{T}$, its classifying topos $\mathcal{E}_\mathbb{T}$ and universal model $U_\mathbb{T}$ have the following classifying property [4, pp. 3–5]:

$$\text{Geom}(\mathcal{E}, \mathcal{E}_\mathbb{T}) \simeq \mathbb{T}\text{-Mod}(\mathcal{E}), \quad f \mapsto f^* U_\mathbb{T}.$$

The equivalence is natural in every Grothendieck topos $\mathcal{E}$. A **Grothendieck topos** is a category of sheaves on a site, up to equivalence; a site is a category with a notion of covering. A **geometric morphism** $f : \mathcal{E} \rightarrow \mathcal{F}$ is an adjunction $f^* \dashv f_*$ with f^* preserving finite limits.

For a geometric morphism $g : \mathcal{E}' \rightarrow \mathcal{E}$, naturality means that the following square commutes up to natural isomorphism:

$$\begin{array}{ccc} \mathrm{Geom}(\mathcal{E}, \mathcal{E}_{\mathbb{T}}) & \xrightarrow{f \mapsto f^* U_{\mathbb{T}}} & \mathbb{T}\text{-Mod}(\mathcal{E}) \\ (-) \circ g \downarrow & & \downarrow g^* \\ \mathrm{Geom}(\mathcal{E}', \mathcal{E}_{\mathbb{T}}) & \xrightarrow{f \mapsto f^* U_{\mathbb{T}}} & \mathbb{T}\text{-Mod}(\mathcal{E}'). \end{array}$$

A **point** of a topos $\mathcal{F}$ is a geometric morphism $\mathbf{Set} \rightarrow \mathcal{F}$. Thus ordinary set-based models correspond, up to the displayed equivalence, to points of $\mathcal{E}_{\mathbb{T}}$; models internal to another topos correspond to generalized points. This statement applies directly to geometric theories. A classical first-order theory can first be Morleyized to a coherent, hence geometric, theory while retaining the same set-based models up to canonical expansion and reduct; internal models then have the semantics of that Morleyization.

The satisfaction pairing also generalizes. Let A be the carrier of an internal $\mathbb{T}$ -model in a Grothendieck topos $\mathcal{E}$, let V be an object of $\mathcal{E}$, and put $X = \{1, \dots, n\}$. A generalized assignment is an arrow $@ : V \rightarrow A^n$. A geometric formula φ in the context X is interpreted by a subobject $\llbracket \varphi \rrbracket \hookrightarrow A^n$, and its locus of satisfaction is the pullback $V_{\varphi} \hookrightarrow V$:

$$\begin{array}{ccc} V_{\varphi} & \xrightarrow{\quad} & \llbracket \varphi \rrbracket \\ \downarrow & \lrcorner & \downarrow \\ V & \xrightarrow{\quad @ \quad} & A^n. \end{array}$$

For $\mathcal{E} = \mathbf{Set}$ and $V = 1$, the subobject $V_{\varphi} \hookrightarrow 1$ is either false or true, recovering the **2**-valued pairing used earlier. In a general topos, the satisfaction locus is a subobject of the stage V ; global truth values are arrows $1 \rightarrow \Omega_{\mathcal{E}}$.

12.2 A spectrum example

The comparison with algebraic geometry can be made exact. Let R be a commutative ring. Introduce one proposition $D(f)$ for each $f \in R$ and impose the geometric sequents [5]

$$D(0) \vdash \perp, \quad \top \vdash D(1), \quad D(fg) \dashv\vdash D(f) \wedge D(g), \quad D(f+g) \vdash D(f) \vee D(g).$$

A set-based model selects the elements f for which $D(f)$ holds. This set is a prime anti-ideal; classically, its complement is a prime ideal. The classifying locale of this propositional geometric theory is the Zariski locale underlying $\mathrm{Spec} R$; its classifying topos is the associated localic topos $\mathbf{Sh}(\mathrm{Spec} R)$. A **locale** describes a space through its lattice of open sets. Thus, when R is the coordinate ring of an affine curve, set-based models correspond bijectively, classically, to the scheme-theoretic points of the curve, including its non-closed points. A prime $\mathfrak{p}$ also determines the field-valued point $\mathrm{Spec} \kappa(\mathfrak{p}) \rightarrow \mathrm{Spec} R$ [6]. This is an instance of models as points.

13 Ultraproducts, stalks, and the @-calculus

The distinction between term evaluation and satisfaction persists under ultraproducts. Prefix @ calculates a term's value in the ultraproduct; postfix @ expresses the transfer of satisfaction in Łoś's theorem. The interpretation of an ultraproduct as a sheaf stalk then realizes satisfaction as membership in a clopen subset of an extremally disconnected space. These constructions also distinguish evaluation of a fixed finite expression from the formation of an ultraproduct of syntax sets.

13.1 Assignments and term evaluation

Work in classical set theory with choice. Let I be a set, and let $\mathcal{U}$ be an ultrafilter on I : a collection of subsets closed under supersets and finite intersections, containing I and excluding $\emptyset$, such that exactly one of B and $I \setminus B$ belongs to $\mathcal{U}$ for each $B \subseteq I$. Let $(\mathcal{A}_i)_{i \in I}$ be nonempty structures in one finitary first-order language $\mathcal{L}$, with functional signature Σ and underlying sets A_i .

Put $C = \prod_{i \in I} A_i$. For $a, b \in C$, define $a \sim_{\mathcal{U}} b$ when $\{i : a_i = b_i\} \in \mathcal{U}$. The ultraproduct $\mathcal{A}_{\mathcal{U}} = \prod_{\mathcal{U}} \mathcal{A}_i$ has underlying set $A_{\mathcal{U}} = C / \sim_{\mathcal{U}}$. Write $q_{\mathcal{U}} : C \rightarrow A_{\mathcal{U}}$ for the quotient map and $[a]_{\mathcal{U}} = q_{\mathcal{U}}(a)$. Function symbols act on representatives coordinatewise; a relation holds of classes precisely when it holds of their representatives on a member of $\mathcal{U}$. The ultrafilter's closure under finite intersections makes these interpretations independent of representatives.

Fix a finite set X of variables and assignments $@_i : X \rightarrow A_i$. Their product is the assignment $@_C : X \rightarrow C$ given by $@_C(x) = (@_i(x))_{i \in I}$. The induced ultraproduct assignment is $@_{\mathcal{U}} = q_{\mathcal{U}} \circ @_C : X \rightarrow A_{\mathcal{U}}$. For $t \in T_{\Sigma}X$, put $e(t) = (@_i t)_{i \in I} \in C$.

The term-evaluation triangle is

$$\begin{array}{ccc} T_{\Sigma}X & \xrightarrow{e} & C \\ & \searrow \llbracket - \rrbracket_{@_{\mathcal{U}}}^{A_{\mathcal{U}}} & \downarrow q_{\mathcal{U}} \\ & & A_{\mathcal{U}}. \end{array}$$

Its calculational form is

$$\boxed{@_{\mathcal{U}} t = [(@_i t)_{i \in I}]_{\mathcal{U}}.}$$

Indeed, the equation for a variable is the definition of $@_{\mathcal{U}}$. If it holds for $t_1, \dots, t_n$, the coordinatewise interpretation of $f \in \Sigma$ proves it for $f(t_1, \dots, t_n)$. This is structural induction on t , including the nullary case. Equivalently, $q_{\mathcal{U}}$ is a homomorphism of the functional reducts, and homomorphisms commute with term evaluation.

13.2 Satisfaction, witnesses, and substitution

For $\varphi \in \text{Form}_{\mathcal{L}}(X)$, define its set of satisfying indices by $B_{\varphi} = \{i \in I : \varphi @_i\}$. Let $u_{\mathcal{U}} : \mathcal{P}(I) \rightarrow \mathbf{2}$ take the value $\top$ exactly on the members of $\mathcal{U}$. It preserves finite Boolean operations: intersections represent conjunction, complements represent negation, and unions represent disjunction.

Łoś's theorem [7, Corollary 10] states that the following square commutes:

$$\begin{array}{ccc} \text{Form}_{\mathcal{L}}(X) & \xrightarrow{\varphi \mapsto B_\varphi} & \mathcal{P}(I) \\ S_{@_{\mathcal{U}}}^{\mathcal{A}_{\mathcal{U}}} \downarrow & & \downarrow u_{\mathcal{U}} \\ \mathbf{2} & \xrightarrow{1_{\mathbf{2}}} & \mathbf{2}. \end{array}$$

In the postfix notation, this is

$$\boxed{\varphi @_{\mathcal{U}} \iff \{i \in I : \varphi @_i\} \in \mathcal{U}.}$$

Here is a proof in the assignment calculus. Atomic equality follows from the term-evaluation identity and the definition of $\sim_{\mathcal{U}}$; the other atomic cases follow from the interpretation of relation symbols. Finite Boolean connectives follow from the corresponding properties of $u_{\mathcal{U}}$.

For the existential step, suppose $B = \{i : ((\exists x)\psi) @_i\} \in \mathcal{U}$. Choose $b_i \in A_i$ such that $\psi(@_i|_{b_i}^x)$ for each $i \in B$, and choose an arbitrary $b_i \in A_i$ elsewhere. Put $b = [(b_i)]_{\mathcal{U}}$. The updated assignments $@_i|_{b_i}^x$, on the context $X \cup \{x\}$, induce exactly $@_{\mathcal{U}}|_b^x$. The induction hypothesis applied to ψ therefore yields $\psi(@_{\mathcal{U}}|_b^x)$, and hence $((\exists x)\psi) @_{\mathcal{U}}$. Conversely, represent a witness $b \in A_{\mathcal{U}}$ by a tuple (b_i) . The induction hypothesis says that $\{i : \psi(@_i|_{b_i}^x)\} \in \mathcal{U}$; this set is contained in B , so $B \in \mathcal{U}$. Universal quantification follows from $(\forall x)\psi \equiv \neg(\exists x)\neg\psi$.

The same update calculation combines transfer with the substitution lemma. For $x \in X$, $s \in T_{\Sigma}X$, and capture-avoiding substitution, it yields

$$\begin{aligned} (\varphi|_s^x) @_{\mathcal{U}} &\iff \varphi(@_{\mathcal{U}}|_{@_{\mathcal{U}}s}^x) \\ &\iff \{i : \varphi(@_i|_{@_i s}^x)\} \in \mathcal{U}. \end{aligned}$$

The first step moves syntactic substitution to the assignment; the second applies Łoś's theorem to the family of updated assignments.

13.3 Ultraproducts as stalks and truth as a locus

Regard I as a discrete space. Its Stone–Čech compactification βI is the space of ultrafilters on I , with clopen basis $\widehat{B} = \{\mathcal{V} : B \in \mathcal{V}\}$, for $B \subseteq I$. The map $B \mapsto \widehat{B}$ identifies $\mathcal{P}(I)$ with the Boolean algebra of clopen subsets of βI . An index i corresponds to the principal ultrafilter $\{B \subseteq I : i \in B\}$.

Let $j : I \hookrightarrow \beta I$ be this inclusion. The family (A_i) is a sheaf on I , whose sections over $B \subseteq I$ are $\prod_{i \in B} A_i$. Its direct image $\mathcal{F}$ on βI satisfies $\mathcal{F}(\widehat{B}) = \prod_{i \in B} A_i$, with restriction maps given by coordinate projections. Consequently,

$$\mathcal{F}_{\mathcal{U}} = \varinjlim_{B \in \mathcal{U}} \prod_{i \in B} A_i \cong A_{\mathcal{U}},$$

where the colimit is taken along restriction to smaller members of $\mathcal{U}$. Two local sections have the same germ precisely when they agree after restriction to a common member of $\mathcal{U}$. Nonemptiness and choice allow a tuple on such a member to extend to all of I , giving the displayed identification with the quotient ultraproduct [7, Example 14].

The assignments $@_i$ determine $@_{\beta} : 1 \rightarrow \mathcal{F}^X$ in $\mathbf{Sh}(\beta I)$, where 1 is the terminal sheaf. Its stalk at $\mathcal{U}$ is $@_{\mathcal{U}}$. Term evaluation yields the global section $@_{\beta} t = (@_i t)_i$. The germ map $g_{\mathcal{U}} : \mathcal{F}(\beta I) \rightarrow \mathcal{F}_{\mathcal{U}}$ therefore satisfies $g_{\mathcal{U}}(@_{\beta} t) = @_{\mathcal{U}} t$. Prefix $@$ returns a section before taking the stalk and an element afterwards.

For the chosen assignments, Łoś's theorem also identifies the locus of ultrafilters satisfying a fixed first-order formula:

$$\{\mathcal{V} \in \beta I : \varphi @_{\mathcal{V}}\} = \widehat{B_{\varphi}}.$$

This has the internal satisfaction-locus interpretation for coherent formulas, built from atomic formulas using finite conjunctions, finite disjunctions, and existential quantification. Interpret the structures as a sheaf model $\mathcal{F}$; let $\llbracket \varphi \rrbracket_{\mathcal{F}} \hookrightarrow \mathcal{F}^X$ be the interpreted relation. For an open subset $O \subseteq \beta I$, write $1_O \hookrightarrow 1$ for its associated subterminal sheaf: its sections over an open W form a singleton if $W \subseteq O$ and are empty otherwise. Then the following square is a pullback:

$$\begin{array}{ccc} 1_{\widehat{B_{\varphi}}} & \longrightarrow & \llbracket \varphi \rrbracket_{\mathcal{F}} \\ \downarrow & \lrcorner & \downarrow \\ 1 & \xrightarrow{\quad @_{\beta} \quad} & \mathcal{F}^X. \end{array}$$

Indeed, direct image along j preserves finite limits, finite coproducts, and images in this situation, so it preserves coherent interpretation [8, Corollary 1.10]. For arbitrary first-order formulas, the same description applies to their predicates in the canonical coherent Morleyization. At a general stage V of an internal model, satisfaction remains a subobject $V_{\varphi} \hookrightarrow V$; at a set-valued point it reduces to the postfix assertion $\varphi @$.

The space βI is compact Hausdorff and extremally disconnected: closures of open subsets are open. Such spaces form a defining site for condensed mathematics. With size bounds fixed, condensed sets can be described as contravariant functors on these spaces that take finite disjoint unions to products [9, Example 2.5 and Proposition 2.8]. For a coherent theory $\mathbb{T}$, the analogous assignment

$$\mathbf{Mod}_{\mathbb{T}}(K) = \mathbb{T}\text{-Mod}(\mathbf{Sh}(K))$$

defines a condensed category of models: inverse image determines its contravariant dependence on K , and disjoint unions of spaces yield products of model categories. Its value at a point is the ordinary category of set-based models. An indexed family of $\mathbb{T}$ -models extends to an object over βI , and its value at an ultrafilter point is the corresponding ultraproduct [8, Example 0.2 and Proposition 1.6]. Here ordinary sheaves on each K supply the values of the condensed category as K varies.

13.4 Finite fields and an infinite disjunction

Let P be the set of primes and take $\mathcal{A}_p = \mathbb{F}_p$. Use $@_p : \emptyset \rightarrow \mathbb{F}_p$ for the empty assignment. For $m \geq 1$, let $\chi_m = (m \cdot 1 = 0)$, where $m \cdot 1$ abbreviates the term formed by adding 1 to itself m times. Then $B_{\chi_m} = \{p \in P : p \mid m\}$ is finite.

A nonprincipal ultrafilter contains no finite set: if a finite union of singletons belongs to an ultrafilter, one of those singletons belongs to it, making it principal. Hence $\neg(\chi_m @_{\mathcal{U}})$ for every $m \geq 1$ and every nonprincipal $\mathcal{U}$ on P . The ultraproduct is a field by Łoś's theorem, and these inequalities show that its characteristic is zero.

Now form the infinitary geometric sentence $\psi = \bigvee_{m \geq 1} \chi_m$. Every $\psi @_p$ is true, since $\chi_p @_p$ is true. But $\psi @_{\mathcal{U}}$ is false at every nonprincipal ultrafilter: none of its disjuncts holds there. The first-order transfer theorem does not include this infinite disjunction.

The sheaf interpretation makes the difference explicit. Each $\widehat{B_{\chi_m}}$ consists of the finitely many principal points indexed by primes dividing m . The satisfaction locus of the geometric disjunction in $\mathbf{Sh}(\beta P)$ is therefore

$$\bigcup_{m \geq 1} \widehat{B_{\chi_m}} = P \subset \beta P.$$

This open subset is dense and proper, and so is not clopen. Density follows because every nonempty basic clopen meets P ; properness follows from the existence of nonprincipal ultrafilters. Taking a stalk preserves the geometric disjunction, which is false at every point of $\beta P \setminus P$. The direct-image functor

j_* , however, does not preserve this infinite disjunction: the disjunction is everywhere true on P , whose terminal sheaf has direct image the terminal sheaf on βP . Thus a satisfaction locus can carry more information than a single external truth value.

13.5 Fixed terms and ultraproducts of syntax

The prefix evaluation identity uses one fixed term $t \in T_\Sigma X$. The comparison between forming terms and forming ultraproducts has a different type:

$$\lambda_{\mathcal{U}} : T_\Sigma \left(\prod_{\mathcal{U}} A_i \right) \longrightarrow \prod_{\mathcal{U}} T_\Sigma A_i.$$

To construct it, give the target its coordinatewise Σ -algebra structure and send a variable $[a]_{\mathcal{U}}$ to $[(\eta_{A_i}(a_i))_i]_{\mathcal{U}}$. This variable map is well-defined, since representatives agreeing on a member of $\mathcal{U}$ have corresponding variable terms agreeing there. The free-algebra property extends it uniquely to $\lambda_{\mathcal{U}}$. Since a term has only finitely many variable occurrences, its image can be computed by choosing representatives of their labels in $A_{\mathcal{U}}$ and keeping the same term shape in every coordinate.

The comparison need not be surjective. Let $I = \mathbb{N}$, let $\mathcal{U}$ be nonprincipal, let every $A_i = \{*\}$, and let Σ consist of one unary symbol f . Then $T_\Sigma(\prod_{\mathcal{U}} A_i) = T_\Sigma\{*\}$ consists of the terms $f^n(*)$, for $n \in \mathbb{N}$, and $\lambda_{\mathcal{U}}(f^n(*)) = [(f^n(*))_i]_{\mathcal{U}}$. The target also contains

$$\tau = [(f^i(*))_{i \in \mathbb{N}}]_{\mathcal{U}}.$$

For each fixed n , the equality $f^i(*) = f^n(*)$ holds only when $i = n$, so $\tau \neq \lambda_{\mathcal{U}}(f^n(*))$. Its representatives have unbounded term depth. Thus the ultraproduct of term sets contains elements that are not represented by any ordinary finite term over the ultraproduct. This does not affect the evaluation identity for a fixed term.

13.6 Names in a common expanded language

The expansion by names also applies, using a common signature for all factors. Retain $C = \prod_i A_i$, and add a constant $\bar{c}$ for each $c \in C$, obtaining $\mathcal{L}_C$. Interpret $\bar{c}$ in $\mathcal{A}_i^C$ as c_i , and in $\mathcal{A}_{\mathcal{U}}^C$ as $[c]_{\mathcal{U}}$. These are the expansions determined by the maps $C \rightarrow A_i$ and $q_{\mathcal{U}} : C \rightarrow A_{\mathcal{U}}$, respectively. The expanded ultraproduct is precisely $\mathcal{A}_{\mathcal{U}}^C = \prod_{\mathcal{U}} \mathcal{A}_i^C$.

Write $@_{\emptyset, \mathcal{A}_i^C} : \emptyset \rightarrow A_i$ and $@_{\emptyset, \mathcal{A}_{\mathcal{U}}^C} : \emptyset \rightarrow A_{\mathcal{U}}$ for the empty assignments in these expanded structures. The name-evaluation calculation is

$$@_{\emptyset, \mathcal{A}_{\mathcal{U}}^C} \bar{c} = [(@_{\emptyset, \mathcal{A}_i^C} \bar{c})_i]_{\mathcal{U}} = [c]_{\mathcal{U}}.$$

Different names $\bar{c}, \bar{d}$ have the same value precisely when $c \sim_{\mathcal{U}} d$; they remain different symbols when $c \neq d$.

The product assignment $@_C : X \rightarrow C$ determines the common ground-term substitution $\overline{@}_C = \nu_C \circ @_C : X \rightarrow T_{\Sigma[C]}\emptyset$. Its value at x is the name of the tuple $(@_i(x))_i$. For an original term t and formula φ , regarded in $\mathcal{L}_C$, the naming identities are

$$\begin{aligned} @_{\mathcal{U}} t &= @_{\emptyset, \mathcal{A}_{\mathcal{U}}^C} (t[\overline{@}_C]), \\ \varphi @_{\mathcal{U}} &\iff \varphi[\overline{@}_C] @_{\emptyset, \mathcal{A}_{\mathcal{U}}^C}. \end{aligned}$$

Evaluating the inserted name for x yields $[(@_i(x))_i]_{\mathcal{U}} = @_{\mathcal{U}}(x)$; the two identities therefore follow from the term and formula substitution lemmas.

13.7 Occurrence maps and transferred truth labels

Fix the finite occurrence-substitution map $\gamma := \gamma_{M,\vec{\sigma}} : D_{M,\vec{n}} \rightarrow C_{\vec{\ell}}$ of Section 9. Let $e_i : C_{\vec{\ell}} \rightarrow \mathbf{2}$ label the formal arguments by their truth values in $\mathcal{A}_i$ under $@_i$, and let $d_i = e_i \circ \gamma$ label their occurrences. Define $e_{\mathcal{U}}$ and $d_{\mathcal{U}}$ by

$$\begin{aligned} e_{\mathcal{U}}(c) = \top &\iff \{i : e_i(c) = \top\} \in \mathcal{U}, \\ d_{\mathcal{U}}(r) = \top &\iff \{i : d_i(r) = \top\} \in \mathcal{U}. \end{aligned}$$

For every r , the two sets $\{i : d_i(r) = \top\}$ and $\{i : e_i(\gamma(r)) = \top\}$ are equal. Hence $d_{\mathcal{U}} = e_{\mathcal{U}} \circ \gamma$. When $e_i(c) = \top$ means $\psi_c @_i$ for a fixed formula ψ_c , Łoś's theorem identifies $e_{\mathcal{U}}(c) = \top$ with $\psi_c @_{\mathcal{U}}$. Thus the occurrence map transports exactly the truth labels supplied by postfix $@$ in the ultraproduct. Evaluation of the fixed finite connective pattern then commutes with $u_{\mathcal{U}}$.

14 Conclusion

The principal objects and maps that elementary notation can obscure are now separated by type:

$$\begin{aligned} \text{terms over } X &: T_{\Sigma}X, \\ \text{syntactic substitutions} &: X \rightarrow T_{\Sigma}X, \\ \text{assignments} &: X \rightarrow A, \\ \text{term evaluation} &: A^X \times T_{\Sigma}X \rightarrow A, \\ \text{formulas over } X &: \text{Form}_{\mathcal{L}}(X), \\ \text{satisfaction} &: \text{Form}_{\mathcal{L}}(X) \times A^X \rightarrow \mathbf{2}. \end{aligned}$$

Categorically, substitution is composition in $\mathbf{Kl}(T_{\Sigma})$, evaluation is the action of an Eilenberg–Moore algebra, and the substitution lemma is their compatibility law. Language expansion turns assignments into ground-term substitutions, with the unit law ensuring that each name evaluates to the element it names. The Došen–Petrić occurrence maps record generalized composition of formula patterns by tracking repeated argument occurrences through finite coproduct fibres and their ordinal encodings.

The parallel $@$ -calculus writes the same maps as calculations: $@t$ returns a term's value, $\varphi @$ records satisfaction, and the sliding identities calculate compatibility with substitution. The two presentations have complementary jobs: categorical maps expose types and compositional laws, while the $@$ -calculus supplies the working notation.

The categorical organization is also functorial. Lawvere theories admit models in any category with finite products, and classifying toposes represent models of geometric theories by points or generalized points. Thus the set-based account is one instance of the construction.

A Feferman’s arithmetization conventions

Arithmetization adds another distinction to those of the preceding sections: an expression, its numerical code, and an arithmetic term denoting that code have different types. Numeral substitution combines all three. The evaluation identity in Appendix A.4 connects this construction with the prefix @-calculus.

In the formula displays of this appendix, $\varphi(v)$ designates a formula with the indicated free-variable slot, following the textbook convention described in Section 7; $\varphi(\bar{n})$ denotes its numeral instance. Dotted occurrences inside quotation marks will be defined by an explicit translation into arithmetic terms.

Feferman represents operations on formal syntax by terms and formulas of arithmetic. We explain the conventions in his *Arithmetization of metamathematics in a general setting* (1960), especially §4, pp. 57–58 [10], and *Transfinite recursive progressions of axiomatic theories* (1962), §2(c), pp. 268–270 [11].

His notation identifies expressions with their Gödel numbers and marks represented operations by dots. A dotted variable abbreviates the represented numeral constructor; combining it with substitution produces the code of a numeral instance. The exposition distinguishes expressions, their numerical codes, and the arithmetic terms denoting those codes. The explicit grammar and coding examples below support this reading of Feferman’s notation. The final section explains its use in Di Paola and Montagna’s parameterized consistency formulas.

A.1 Numbers, numerals, and the function Num

The arithmetic language

The first-order language of arithmetic used here has nonlogical signature

$$L_A = \{0, S, +, \cdot\},$$

where 0 is a constant symbol, S is unary, and $+$ and $\cdot$ are binary function symbols. Equality is a logical symbol. With variables $v_0, v_1, \dots$, its terms t and formulas A are generated by

$$\begin{aligned} t &::= v_i \mid 0 \mid S(t) \mid t + t \mid t \cdot t, \\ A &::= t = t \mid \neg A \mid A \rightarrow A \mid \forall v_i A. \end{aligned}$$

Parentheses specify the grouping of expressions. Other connectives and quantifiers are abbreviations in this syntax. The symbol $\cdot$ is multiplication; the dot above a variable has a different role.

The corresponding standard structure is

$$\mathcal{N} = \langle \mathbb{N}, 0^{\mathcal{N}}, S^{\mathcal{N}}, +^{\mathcal{N}}, \cdot^{\mathcal{N}} \rangle, \quad S^{\mathcal{N}}(n) = n + 1,$$

with addition and multiplication of natural numbers. Thus the signature lists symbols and their arities, whereas the structure also specifies a domain and interpretations. Below, $\mathbb{N}$ also denotes this structure and its expansion by the specified primitive recursive functions.

Fix a language L^* extending L_A by function symbols for the primitive recursive functions needed below. Fix an injective Gödel coding $E \mapsto \#(E)$ of its finite expressions, with primitive recursive syntax predicates and constructors. Expressions are coded as raw syntax: distinct bound-variable names remain distinct, although Section 7 takes formulas modulo α -equivalence. Provably equal terms and logically equivalent formulas also retain their distinct codes. All displayed formula abbreviations are expanded by fixed conventions before coding.

Numerals and their codes

For $n \in \mathbb{N}$, its *numeral* is the closed term defined by

$$\bar{0} := 0, \quad \overline{n+1} := S(\bar{n}).$$

Thus $\bar{2}$ is the term $S(S(0))$. Throughout this appendix, $\bar{n}$ denotes this numeral, whereas the bar in Section 10 denotes an adjoined constant symbol naming an element. No new constant symbols are needed for these numerals. Define the numerical function

$$\boxed{\text{num}(n) := \#(\bar{n})}. \quad (\text{A.1})$$

Its input is a number; its output is another number, namely a code. If $\text{sc}(\#(t)) = \#(S(t))$ for each term t , with $\text{sc}(a) = \#(0)$ on non-term codes, then

$$\text{num}(0) = \#(0), \quad \text{num}(n+1) = \text{sc}(\text{num}(n)). \quad (\text{A.2})$$

These equations prove that num is primitive recursive. In particular, $\text{num}(n+1)$ is obtained by coding an additional successor symbol; it is not defined as $\text{num}(n) + 1$.

Write PA^* for a conservative definitional extension of PA containing the chosen primitive recursive function symbols and their defining equations. The symbol Num represents num , and Sc represents sc . Their equations include

$$\text{PA}^* \vdash \text{Num}(0) = \overline{\#(0)}, \quad \text{PA}^* \vdash \forall x \text{Num}(Sx) = \text{Sc}(\text{Num}(x)).$$

For every standard n ,

$$\text{PA}^* \vdash \text{Num}(\bar{n}) = \overline{\text{num}(n)}. \quad (\text{A.3})$$

Lower-case num denotes the numerical function in the metatheory; upper-case Num is the arithmetic function symbol. When this distinction is suppressed, both are often written Num .

The construction factors through the set of numeral expressions:

$$\begin{array}{ccc} \mathbb{N} & \xrightarrow{n \mapsto \bar{n}} & \text{Numerals}(L^*) \\ & \searrow \text{num} & \downarrow \# \\ & & \mathbb{N}. \end{array}$$

The following expressions have different syntactic roles.

Expression	Meaning
n	A natural number in the metatheory.
$\bar{n}$	An arithmetic term denoting n .
$\text{num}(n)$	The natural number $\#(\bar{n})$.
$\text{Num}(x)$	An arithmetic term with free variable x .
$\text{Num}(\bar{n})$	A closed arithmetic term denoting $\text{num}(n)$.
$\overline{\text{num}(n)}$	The numeral denoting $\text{num}(n)$.

For example, $\text{Num}(\bar{2}) = \text{Num}(S(S(0)))$ and $\overline{\text{num}(2)}$ have the same value, but they are different terms and therefore have different Gödel codes.

Feferman's name for the numerical constructor is nm . The dot convention for represented functions appears in [10, §4, p. 57], with the numeral constructor on p. 58. The primitive recursive extensions

are introduced in [10, §3, Definition 3.3 and Theorem 3.4, pp. 52–53]. In [11, §2(c), p. 269], Feferman explicitly abbreviates the represented numeral constructor applied to u by $\dot{u}$. In our notation,

$$\dot{u} \text{ abbreviates } \text{Num}(u). \quad (\text{A.4})$$

This is a special case of the general dot convention, not a separate operation on variables. Feferman's description of the output as the $(u + 1)$ -st numeral counts $\bar{0}$ as the first numeral.

Concrete values of $\#$

The number $\#(E)$ depends on the chosen coding. For explicit numerical examples, use the following coding of syntax trees. These numerical assignments are chosen here; they are not Feferman's particular numbering. The remaining definitions and results apply to any coding with the primitive recursive operations already specified.

Define the pairing function and the constructor-code function by

$$\pi(a, b) := \frac{(a + b)(a + b + 1)}{2} + b, \quad c(j, a) := 1 + \pi(j, a).$$

The first argument j of c records which syntactic constructor is used. Code terms and formulas recursively as follows:

$$\begin{aligned} \#(0) &= c(0, 0), & \#(v_i) &= c(1, i), \\ \#(S(t)) &= c(2, \#(t)), & \#(t + u) &= c(3, \pi(\#(t), \#(u))), \\ \#(t \cdot u) &= c(4, \pi(\#(t), \#(u))), & \#(t = u) &= c(5, \pi(\#(t), \#(u))), \\ \#(\neg A) &= c(6, \#(A)), & \#(A \rightarrow B) &= c(7, \pi(\#(A), \#(B))), \\ \#(\forall v_i A) &= c(8, \pi(i, \#(A))). \end{aligned}$$

Injectivity follows by recovering the constructor and its arguments from the pairing. Parentheses determine the syntax tree and are not separately coded. The coding extends to each added function symbol f_j of arity r by the rule

$$\#(f_j(t_1, \dots, t_r)) = c(9 + j, \sigma(\#(t_1), \dots, \#(t_r))),$$

where the finite-sequence code satisfies $\sigma() = 0$ and $\sigma(a_1, \dots, a_r) = 1 + \pi(a_1, \sigma(a_2, \dots, a_r))$ for $r > 0$. The added symbols are indexed with primitive recursive arity lookup.

For this coding the following are actual natural-number values of $\#$:

Expression E	$\#(E)$
0	1
v_0	2
$S(0)$	8
$S(S(0))$	64
$0 + 0$	33
$0 = 0$	50

For example,

$$\#(S(0)) = 1 + \pi(2, 1) = 8, \quad \#(0 = 0) = 1 + \pi(5, \pi(1, 1)) = 50.$$

Consequently $\text{num}(0) = 1$, $\text{num}(1) = 8$, and $\text{num}(2) = 64$; in particular, $\text{PA}^* \vdash \text{Num}(\bar{2}) = \bar{64}$. The terms 0 and $0 + 0$ both denote zero, but their codes are 1 and 33.

A.2 Quotation and substitution

For a fixed expression E , its *ordinary Gödel quotation* is the numeral

$$\ulcorner E \urcorner := \overline{\#(E)}. \quad (\text{A.5})$$

It is closed even when E is open. Thus x is not free in $\ulcorner x = x \urcorner$. For the concrete coding above,

$$\ulcorner S(0) \urcorner = \overline{8}, \quad \ulcorner 0 = 0 \urcorner = \overline{50}, \quad \ulcorner v_0 = 0 \urcorner = \overline{86}.$$

The number 86 codes the open formula $v_0 = 0$; its quotation is the closed numeral $\overline{86}$.

For each $k \geq 1$, let

$$\text{sb}_k(a; v_1, \dots, v_k; b_1, \dots, b_k)$$

be the primitive recursive function that simultaneously substitutes the terms with codes b_i for the free occurrences of the distinct variables with codes v_i in the expression with code a . Fix a deterministic procedure for renaming bound variables when needed to avoid capture; preserve the existing names when no renaming is needed. Set the result to 0 on invalid inputs. Let Sb_k be its representing function symbol in L^* , and write Sb for Sb_1 . The semicolons separate blocks of arguments; they are not additional object-language operations. Feferman's corresponding substitution operations are Sb and simultaneous substitution SSb [10, §2(b), pp. 42–43; §4, p. 57].

For a fixed expression E and a designated variable v , define

$$q_{E;v}(t) := \text{Sb}(\ulcorner E \urcorner; \ulcorner v \urcorner; \text{Num}(t)). \quad (\text{A.6})$$

This is an arithmetic term. Its value when t has value n is

$$\#(E[\overline{n}/v]),$$

where the bracket denotes replacement of the free occurrences of v . The inserted expression is a numeral, so it has no variables that could be captured. When $E = \varphi(v)$ and v is understood, write $q_\varphi(t)$ for $q_{\varphi;v}(t)$.

The conventional notation

$$\ulcorner \varphi(\dot{x}) \urcorner := q_\varphi(x) = \text{Sb}(\ulcorner \varphi(v) \urcorner; \ulcorner v \urcorner; \text{Num}(x)) \quad (\text{A.7})$$

therefore denotes the code of an instance, as a function of x . In contrast, $\ulcorner \varphi(v) \urcorner$ is the fixed numeral for the code of the open formula. Feferman's substitution abbreviation on p. 269 of [11] is precisely this numeral-substitution construction, in his convention identifying expressions with their codes.

A.3 Making numeral substitution explicit

To make the substitution conventions explicit, the following auxiliary grammar designates the occurrences to be filled with numerals. Its translation uses Feferman's represented numeral constructor and substitution operations. The occurrence syntax is supplied for this exposition: it distinguishes ordinary variables in the quoted expression from the arithmetic parameters that determine the inserted numerals.

Templates and arithmetic expressions

Let v range over object variables, and let $\mathbf{h}_1, \mathbf{h}_2, \dots$ be distinct *hole symbols*, which are not variables and cannot be bound. A term template U and formula template A have grammar

$$\begin{aligned} U &::= v \mid 0 \mid S(U) \mid U + U \mid U \cdot U \mid f(U_1, \dots, U_r) \mid \mathbf{h}_i, \\ A &::= U = U \mid \neg A \mid A \rightarrow A \mid \forall v A. \end{aligned}$$

Here f ranges over the chosen function symbols of L^* , with their assigned arities. A template C is either kind. It contains no quotation constructor. Connectives such as $\wedge, \vee, \exists$, and formula notations such as Pr_T and Con_α , are fixed abbreviations, expanded before a template is coded.

Surface arithmetic terms t and formulas B have grammar

$$\begin{aligned} t &::= x \mid 0 \mid S(t) \mid t + t \mid t \cdot t \mid f(t_1, \dots, t_r) \mid \mathbf{Q}(C; t_1, \dots, t_k), \\ B &::= t = t \mid \neg B \mid B \rightarrow B \mid \forall x B. \end{aligned}$$

The distinct holes occurring in C must be exactly $\mathbf{h}_1, \dots, \mathbf{h}_k$; a hole may occur more than once. When $k = 0$, write $\mathbf{Q}(C;)$.

Translation into the arithmetic language

Write t^b for the translation of a surface term into L^* . Translate variables and arithmetic constructors component by component. For $\mathbf{Q}$, choose distinct variables $w_1, \dots, w_k$ that occur nowhere in C , by a fixed rule. Let C^w replace each hole $\mathbf{h}_i$ by w_i . Set

$$(\mathbf{Q}(C;))^b := \ulcorner C \urcorner, \quad (\text{A.8})$$

$$(\mathbf{Q}(C; t_1, \dots, t_k))^b := \text{Sb}_k(\ulcorner C^w \urcorner; \ulcorner w_1 \urcorner, \dots, \ulcorner w_k \urcorner; \text{Num}(t_1^b), \dots, \text{Num}(t_k^b)) \quad (k > 0). \quad (\text{A.9})$$

Formula translation is likewise componentwise. Equations (A.8)–(A.9) eliminate quotation and holes; the result is an ordinary arithmetic expression.

The free-variable rule is

$$\text{FV}((\mathbf{Q}(C; t_1, \dots, t_k))^b) = \bigcup_{i=1}^k \text{FV}(t_i^b). \quad (\text{A.10})$$

Ordinary variables in C occur inside fixed code numerals and contribute no free variables to the translated term. A quantifier outside $\mathbf{Q}$ can bind variables in its arguments. A quantifier inside C binds only the ordinary object-variable occurrences in its own scope.

Dotted display notation abbreviates this constructor: put $\dot{t}_i$ at the occurrences of $\mathbf{h}_i$, and enclose the resulting display in Gödel corners. The dot records *numeral insertion* at those occurrences. Thus a generalized dotted term uses $\text{Num}(t_i^b)$, not the code of the term t_i itself.

Example A.1 (Ordinary occurrences remain quoted). The display $\ulcorner x = \dot{x} \urcorner$ means $\mathbf{Q}(x = \mathbf{h}_1; x)$. With a fresh w , its translation is

$$\text{Sb}(\ulcorner x = w \urcorner; \ulcorner w \urcorner; \text{Num}(x)).$$

At $x = n$, its value is $\#(x = \bar{n})$, an open-formula code. Replacing the dot by the ordinary variable x and then substituting for every free x in $x = x$ would instead produce $\#(\bar{n} = \bar{n})$. Fresh placeholders prevent this error.

Example A.2 (A hole under a quantifier). The display $\ulcorner \forall x (x = \dot{x}) \urcorner$ means $\mathbf{Q}(\forall x (x = \mathbf{h}_1); x)$. It translates as

$$\text{Sb}(\ulcorner \forall x (x = w) \urcorner; \ulcorner w \urcorner; \text{Num}(x)),$$

and has value $\#(\forall x (x = \bar{n}))$ at $x = n$. The inner $\forall x$ does not bind the parameter supplied to the hole. Consequently this grammar needs no prohibition on a dotted x under an inner $\forall x$. The explicit $\mathbf{Q}$ form resolves the two roles of the same printed letter.

Example A.3 (Repeated holes and compound parameters). The term $\mathbf{Q}(\mathbf{h}_1 = \mathbf{h}_1; x)$ has value $\#(\bar{n} = \bar{n})$ at $x = n$. The term $\mathbf{Q}(\mathbf{h}_1 = \mathbf{h}_2; x, Sx)$ has value $\#(\bar{n} = \overline{n+1})$. For a fixed $\varphi(v)$, $\ulcorner \varphi((x + y)) \urcorner$ denotes $q_\varphi(x + y)$: at $x = n, y = m$ it codes $\varphi(\overline{n+m})$, not $\varphi(\bar{n} + \bar{m})$.

A.4 The substitution lemma

Proposition A.4 (Numeral evaluation). *Let the holes of C be $\mathbf{h}_1, \dots, \mathbf{h}_k$. For each tuple of standard natural numbers $n_1, \dots, n_k$,*

$$\text{PA}^* \vdash (\mathbf{Q}(C; \overline{n_1}, \dots, \overline{n_k}))^b = \ulcorner C[\overline{n_1}/\mathbf{h}_1, \dots, \overline{n_k}/\mathbf{h}_k] \urcorner. \quad (\text{A.11})$$

The right side is an ordinary quotation, hence a numeral.

Proof. For $k = 0$, the assertion is the defining identity (A.8). For $k > 0$, replace each $\text{Num}(\overline{n_i})$ by $\overline{\text{num}(n_i)}$ using (A.3). The remaining closed Sb_k term computes the code obtained by substituting $\overline{n_i}$ for the fresh w_i . Those variables occur exactly where the holes occurred and are not bound. The result is exactly the expression on the right. The defining equations prove this finite computation in PA^* . $\square$

In particular, when $q(x) = (\mathbf{Q}(C; x))^b$,

$$\text{PA}^* \vdash q(x)[\overline{n}/x] = \ulcorner C[\overline{n}/\mathbf{h}_1] \urcorner.$$

The substitution on the left is substitution in the translated arithmetic term. Undotted variable occurrences inside C remain quoted. More generally, if a valuation s assigns t_i^b the value n_i , then

$$\llbracket (\mathbf{Q}(C; t_1, \dots, t_k))^b \rrbracket_s^{\mathbb{N}} = \#(C[\overline{n_1}/\mathbf{h}_1, \dots, \overline{n_k}/\mathbf{h}_k]).$$

With $@ = s$, the prefix evaluation convention of Section 8 writes the same identity as

$$@(\mathbf{Q}(C; t_1, \dots, t_k))^b = \#(C[\overline{@t_1^b}/\mathbf{h}_1, \dots, \overline{@t_k^b}/\mathbf{h}_k]).$$

Here each $@t_i^b$ is a number; its bar constructs a numeral, which fills a hole before the resulting expression is coded. These are semantic identities; (A.11) is the separate provability assertion for each fixed tuple of numerals. The use of PA^* specifies the language in which the displayed function terms exist. Their occurrences can be eliminated to obtain statements in PA .

A.5 Occurrence profiles and arithmetized substitution

The finite occurrence construction of Section 9 applies to the term holes of an arithmetic template. Each hole receives a term. The occurrence map records which argument fills each slot, while the surrounding syntax specifies the constructors of the resulting expression.

Let C be a term or formula template with distinct holes $\mathbf{h}_1, \dots, \mathbf{h}_k$ and m hole occurrences. Number the occurrences from left to right and define

$$o_C : \underline{m} \longrightarrow \underline{k}, \quad o_C(r) = i \iff \text{the } r\text{-th hole occurrence is } \mathbf{h}_i.$$

Let C_{lin} be the template obtained by replacing the r -th hole occurrence by $\mathbf{h}_r$, for every $r \in \underline{m}$. Thus each hole of C_{lin} occurs exactly once. This replacement is performed on the occurrence slots of C in one step. It leaves every ordinary variable, function symbol, connective, and quantifier unchanged.

For any set B , precomposition with o_C defines

$$o_C^* : B^{\underline{k}} \longrightarrow B^{\underline{m}}, \quad o_C^*(b_1, \dots, b_k) = (b_{o_C(1)}, \dots, b_{o_C(m)}).$$

Let $K \subseteq \mathbb{N}$ be the set of codes of closed L^* -terms. Choose the fresh variables $w_1, \dots, w_k$ used in the translation of C and write $C^{\mathbf{w}}$ as in (A.9). Define

$$b_C : K^{\underline{k}} \longrightarrow \mathbb{N}, \quad b_C(\vec{a}) = \text{sb}_k(\#(C^{\mathbf{w}}); \#(w_1), \dots, \#(w_k); a_1, \dots, a_k) \quad (k > 0).$$

For $k = 0$, the domain is a singleton and $b_C() = \#(C)$. Define $b_{C_{\text{lin}}}$ in the same way, using fresh variables $z_1, \dots, z_m$ for its holes. The variables in each list occur nowhere in the corresponding template.

Proposition A.5 (Occurrence factorization). *For every $\vec{a} \in K^k$,*

$$b_C(\vec{a}) = b_{C_{\text{lin}}}(o_C^* \vec{a}). \quad (\text{A.12})$$

Consequently, the following diagram commutes, where the upper vertical maps apply num coordinatewise:

$$\begin{array}{ccc} \mathbb{N}^k & \xrightarrow{o_C^*} & \mathbb{N}^m \\ \text{num}^k \downarrow & & \downarrow \text{num}^m \\ K^k & \xrightarrow{o_C^*} & K^m \\ b_C \downarrow & & \downarrow b_{C_{\text{lin}}} \\ \mathbb{N} & \xlongequal{\quad} & \mathbb{N}. \end{array}$$

Proof. Write $a_i = \#(u_i)$ with u_i a closed term. Substitution into $C^{\mathbf{w}}$ inserts $u_{o_C(r)}$ at the r -th original hole occurrence. Substitution into $C_{\text{lin}}^{\mathbf{z}}$ with the argument list $o_C^*(u_1, \dots, u_k)$ inserts the same term at the same position. Structural recursion on C therefore yields the raw-syntax identity

$$C^{\mathbf{w}}[u_1/w_1, \dots, u_k/w_k] = C_{\text{lin}}^{\mathbf{z}}[u_{o_C(1)}/z_1, \dots, u_{o_C(m)}/z_m].$$

All inserted terms are closed, so no variable can be captured and the specified substitution procedure preserves the bound-variable names. Coding this identity proves (A.12). The upper square commutes coordinatewise: both paths send $\vec{n}$ to $(\text{num}(n_{o_C(r)}))_{r \in \underline{m}}$. When $k = 0$, also $m = 0$, and both lower maps have the constant value $\#(C)$. $\square$

For surface arithmetic terms $t_1, \dots, t_k$ and a current assignment $@$ in the standard arithmetic structure, the diagram and the translation (A.9) yield

$$@(\mathbf{Q}(C; t_1, \dots, t_k))^b = @(\mathbf{Q}(C_{\text{lin}}; t_{o_C(1)}, \dots, t_{o_C(m)}))^b. \quad (\text{A.13})$$

For $m > 0$, the arithmetic term evaluated on the right expands to

$$\text{Sb}_m(\ulcorner C_{\text{lin}}^{\mathbf{z}} \urcorner; \ulcorner z_1 \urcorner, \dots, \ulcorner z_m \urcorner; \text{Num}(t_{o_C(1)}^b), \dots, \text{Num}(t_{o_C(m)}^b)).$$

Thus the occurrence map duplicates and reorders the represented numeral arguments of Feferman's substitution operation. For each fixed tuple of standard numerals, the equality between the two closed translated terms is provable in PA^* by their defining computations, as in Proposition A.4.

Example A.6 (A repeated numeral argument). For $C = \mathbf{h}_1 + \mathbf{h}_1$, the profile is $o_C = (1, 1)$ and $C_{\text{lin}} = \mathbf{h}_1 + \mathbf{h}_2$. The upper horizontal map in the diagram is the diagonal $n \mapsto (n, n)$. In the prefix notation,

$$@(\mathbf{Q}(\mathbf{h}_1 + \mathbf{h}_1; x))^b = \#(\overline{@x} + \overline{@x}).$$

At $@x = 0$, the value is $\#(0 + 0) = 33$ for the coding of Appendix A.1, whereas $\text{num}(0 + 0) = \#(0) = 1$. The code retains the addition symbol. The template $\mathbf{h}_1 \cdot \mathbf{h}_1$ has the same occurrence profile and a different constructor, so its substitution codes are different. The profile and the preserved syntax together determine the operation.

Composition of templates

For each $i \in \underline{k}$, let D_i be a term template on the holes $\mathbf{g}_{i,1}, \dots, \mathbf{g}_{i,\ell_i}$, with no ordinary variables. Assume these hole families are pairwise disjoint. Let D_i have n_i hole occurrences and profile $o_i : \underline{n_i} \rightarrow \underline{\ell_i}$. Replace each $\mathbf{h}_i$ in C by D_i and denote the resulting template by $C[\vec{D}]$. Index its holes in the block order of Section 9.

Each hole occurrence in $C[\vec{D}]$ has coordinates (r, t) , where r selects a hole occurrence of C and t selects an occurrence in the inserted copy of $D_{o_C(r)}$. Its hole label is therefore recorded by exactly the fibrewise map of that section:

$$D_{C,\vec{n}} = \bigoplus_{r=1}^m \underline{n_{o_C(r)}}, \quad C_{\vec{\ell}} = \bigoplus_{i=1}^k \underline{\ell_i},$$

$$\gamma_{C,\vec{\sigma}} : D_{C,\vec{n}} \longrightarrow C_{\vec{\ell}}, \quad \gamma_{C,\vec{\sigma}}(r, t) = (o_C(r), o_{o_C(r)}(t)).$$

An assignment of closed-term codes $a : C_{\vec{\ell}} \rightarrow K$ pulls back along $\gamma_{C,\vec{\sigma}}$ to the occurrence labels

$$(a \circ \gamma_{C,\vec{\sigma}})(r, t) = a(o_C(r), o_{o_C(r)}(t)).$$

The ordinal encodings of Section 9 transport this map to $\delta_{C,\vec{\sigma}}$.

Let $a_i = (a(i, 1), \dots, a(i, \ell_i))$. Filling D_i with closed terms produces a closed term, so $b_{D_i}(a_i) \in K$. The substitution maps on codes form the typed commutative square

$$\begin{array}{ccc} K^{C_{\vec{\ell}}} & \xrightarrow{a \mapsto (b_{D_i}(a_i))_{i \in \underline{k}}} & K^{\underline{k}} \\ b_{C[\vec{D}]} \downarrow & & \downarrow b_C \\ \mathbb{N} & \xlongequal{\quad} & \mathbb{N}. \end{array}$$

Indeed, both paths code the expression obtained by filling every final hole of $C[\vec{D}]$ with the prescribed closed term. Filling the inner templates first and then inserting them in C yields that same raw expression. No bound-variable renaming is required. Equivalently,

$$b_{C[\vec{D}]}(a) = b_C(b_{D_1}(a_1), \dots, b_{D_k}(a_k)). \quad (\text{A.14})$$

The map $\gamma_{C,\vec{\sigma}}$ specifies the occurrence routing for this composition; the primitive recursive substitution functions compute its output code. Numeral insertion into the final holes is the specialization $a(i, s) = \text{num}(n_{i,s})$.

A.6 Three provability assertions

Fix a recursively enumerable theory T in L^* whose nonlogical axioms are sentences, and fix an arithmetized proof predicate that allows open formulas as proof conclusions. Let $\text{Pr}_T(a)$ assert that some finite T -proof has conclusion with code a . Its standard interpretation is the actual theoremhood relation of T . Fix $\varphi(v)$ with v as its only free variable.

Closed arithmetic sentence	What its truth in $\mathbb{N}$ means
$\text{Pr}_T(\ulcorner \forall v \varphi(v) \urcorner)$	$T \vdash \forall v \varphi(v)$.
$\text{Pr}_T(\ulcorner \varphi(v) \urcorner)$	$T \vdash \varphi(v)$, with v free in the conclusion.
$\forall x \text{Pr}_T(q_\varphi(x))$	For every $n \in \mathbb{N}$, $T \vdash \varphi(\bar{n})$.

The first two express equivalent theoremhood conditions: universal elimination proves one direction and generalization proves the other, because v is not free in any axiom. This hypothesis is explicit in [10, §2(c), Proposition 2.1, p. 45]; see also [11, §2(b), p. 266].

The third concerns proofs of numeral instances. Precisely,

$$\mathbb{N} \models \forall x \text{Pr}_T(q_\varphi(x)) \iff (\forall n \in \mathbb{N}) T \vdash \varphi(\bar{n}). \quad (\text{A.15})$$

This follows from the value of $q_\varphi(x)$ at $x = n$ and the correctness of the standard proof predicate. Proposition A.4 also yields, for every fixed n ,

$$\text{PA}^* \vdash \text{Pr}_T(q_\varphi(\bar{n})) \leftrightarrow \text{Pr}_T(\ulcorner \varphi(\bar{n}) \urcorner).$$

This last inference uses equality congruence in arithmetic. It is not a reflection principle.

Equation (A.15) does not say that T proves the displayed universal provability sentence, nor that T proves $\forall v \varphi(v)$. In particular, a family of proofs of numeral instances is not a proof with a free variable to which the generalization rule can be applied.

For example, if $E_{e,f}(v)$ is a fixed formula expressing the equality under consideration, with e, f fixed, the two relations can be defined by

$$\begin{aligned} e \equiv_T f &\iff T \vdash \forall v E_{e,f}(v), \\ e \approx_T f &\iff (\forall n \in \mathbb{N}) T \vdash E_{e,f}(\bar{n}). \end{aligned}$$

The first row of the table expresses the former and the third row the latter, when interpreted in $\mathbb{N}$ with $\varphi = E_{e,f}$. The formula chosen for equality, especially for partial functions, remains part of these definitions.

A.7 Nesting: coding a term and coding its value

Quotation is applied to a fixed arithmetic expression after its inner abbreviations have been expanded. Consequently

$$\ulcorner \text{Pr}_T(\ulcorner \varphi(\dot{x}) \urcorner) \urcorner = \ulcorner \text{Pr}_T(q_\varphi(x)) \urcorner \quad (\text{A.16})$$

is a closed numeral: it codes an open formula. The free x in the inner arithmetic term is part of the syntax coded by the outer quotation.

Two different parameterized constructions are useful. Put

$$F(v) := \text{Pr}_T(q_\varphi(v)), \quad G(v) := \text{Pr}_T(v), \quad m_n := \#(\varphi(\bar{n})).$$

The q notation always designates substitution for the indicated free variable of the fixed, expanded formula. In particular, $q_\varphi(v)$ in the definition of F is its explicit arithmetic term. Then

$$(q_F(x))^{\mathbb{N}, x \mapsto n} = \#(\text{Pr}_T(q_\varphi(\bar{n}))), \quad (\text{A.17})$$

$$(q_G(q_\varphi(x)))^{\mathbb{N}, x \mapsto n} = \#(\text{Pr}_T(\bar{m}_n)). \quad (\text{A.18})$$

The first construction inserts $\bar{n}$ into the syntax of the computation $q_\varphi(v)$; the second inserts the numeral for that computation's *value* into G . Explicitly, the term occurring inside the first coded formula is

$$q_\varphi(\bar{n}) = \text{Sb}(\ulcorner \varphi(v) \urcorner; \ulcorner v \urcorner; \text{Num}(\bar{n})),$$

for the designated variable v of φ . It is not the numeral $\bar{m}_n$. These terms have different syntax. With the fixed proof predicate, whose argument occurs in its expansion, the two coded formulas also have different syntax and hence different codes.

On the other hand, Proposition A.4 yields

$$\text{PA}^* \vdash q_\varphi(\bar{n}) = \overline{m_n}, \quad \text{PA}^* \vdash \text{Pr}_T(q_\varphi(\bar{n})) \leftrightarrow \text{Pr}_T(\overline{m_n}).$$

Provable equality of terms licenses replacement in formulas; it does not identify the codes of the terms or of the resulting formulas. The same distinction already occurs between $\text{Num}(\bar{n})$ and $\overline{\text{num}(n)}$ in Appendix A.1.

For reading nested expressions, expand the numeral-constructor and substitution abbreviations before taking the outer code. The terms $q_F(x)$ and $q_G(q_\varphi(x))$ display the two constructions explicitly. A notation assigning quotation depths to repeated dots would require additional syntactic conventions.

A.8 Axiom parameters and consistency formulas

A formula and a function producing its code

An *axiom formula* $\alpha(u)$, with $\text{FV}(\alpha) \subseteq \{u\}$, defines a set of axiom codes by

$$A_\alpha := \{a \in \mathbb{N} : \mathbb{N} \models \alpha(\bar{a})\}.$$

Assume it selects only codes of sentences. Fix a proof calculus and an arithmetical formula $\text{Pr}_\alpha(a, p)$ saying that p codes a finite derivation with conclusion code a , whose nonlogical axioms satisfy α . Choose this formula by a fixed primitive recursive construction from the syntax of α . Define

$$\text{Pr}_\alpha(a) := \exists p \text{Pr}_\alpha(a, p), \quad \text{Con}_\alpha := \neg \text{Pr}_\alpha(\ulcorner \perp \urcorner), \quad (\text{A.19})$$

where $\perp$ is the fixed logical contradiction $(0 = 0) \wedge \neg(0 = 0)$. This choice also works for fragments not containing arithmetic axioms. Feferman's formulation excludes a formula and its negation both being provable; it is equivalent to (A.19) for this classical proof calculus [11, §2(c), p. 269].

The formula Con_α is distinct from the primitive recursive function taking $\#(\alpha)$ to $\#(\text{Con}_\alpha)$. Relative to the fixed construction in (A.19), denote that numerical function by con^* and its represented function symbol by Con^* . Thus

$$\text{con}^*(\#(\alpha)) = \#(\text{Con}_\alpha), \quad \text{PA}^* \vdash \text{Con}^*(\ulcorner \alpha \urcorner) = \ulcorner \text{Con}_\alpha \urcorner. \quad (\text{A.20})$$

Set the numerical function to 0 on inputs outside its prescribed syntactic domain. Feferman makes exactly the distinction between consistency formulas and represented code-producing functions on p. 269 of [11]. The output Gödel numbers depend on the selected definition of the consistency formula.

Adding a numeral instance as an axiom

Fix a formula $\varphi(v)$ with no free variables other than v , and define

$$\beta(z; u) := \alpha(u) \vee u = q_\varphi(z). \quad (\text{A.21})$$

Choose z distinct from u . Here z is a parameter and u is the variable for axiom codes. For each $n \in \mathbb{N}$, $\beta(\bar{n}; u)$ defines the axiom set

$$A_\alpha \cup \{\#(\varphi(\bar{n}))\}.$$

The parameterized proof and consistency formulas are formed while leaving z free; auxiliary proof variables are chosen fresh. We write the latter as $\text{Con}_{\beta(z; \cdot)}$. This is a formula with parameter z , not a term. The construction is the parameter convention of [11, §2(c), p. 270].

One may abbreviate it by $\text{Con}_{\alpha + \varphi(z)}$, provided that the subscript is defined by (A.21). The plus sign then denotes adjoining an axiom through its axiom formula. It does not assert that $\alpha + \varphi(z)$ is itself an arithmetic term. The arithmetic term selecting the added axiom is $q_\varphi(z)$; a represented constructor such as Con^* is needed to compute the code of a consistency formula.

Adding a convergence sentence

Di Paola and Montagna apply the preceding construction to a finite axiom set together with a single sentence asserting that a program halts [12, pp. 645–646]. Let $\alpha(u)$ select the axiom codes of an arithmetical theory. Fix $n \in \mathbb{N}$ and choose u, z, x pairwise distinct. The bounded axiom formula

$$(\alpha \upharpoonright z)(u) := \alpha(u) \wedge u \leq z \quad (\text{A.22})$$

selects, at $z = n$, the finitely many axiom codes at most n .

Fix a program index e and a primitive recursive computation predicate $\mathcal{T}$. The convergence formula

$$\theta_e(v) := \exists y \mathcal{T}(\bar{e}, v, y), \quad (\text{A.23})$$

abbreviated by $\varphi_e(v) \downarrow$, says that program e halts on input v . For $m \in \mathbb{N}$, the code of $\theta_e(\bar{m})$ is the value at $x = m$ of

$$q_{\theta_e}(x) := \text{Sb}(\ulcorner \theta_e(v) \urcorner; \ulcorner v \urcorner; \text{Num}(x)). \quad (\text{A.24})$$

Thus

$$\beta_e(z, x; u) := (\alpha(u) \wedge u \leq z) \vee u = q_{\theta_e}(x) \quad (\text{A.25})$$

selects, at $z = n$ and $x = m$, the bounded axiom set together with the single sentence $\theta_e(\bar{m})$.

Di Paola and Montagna’s notation [12, p. 646] means

$$\text{Con}_{\alpha \upharpoonright \bar{n} + \varphi_e(\dot{x}) \downarrow} := \text{Con}_{\beta_e(\bar{n}, x; \cdot)}. \quad (\text{A.26})$$

At $x = m$, this formula asserts that no contradiction is derivable from the axioms with codes at most n together with the convergence sentence $\varphi_e(\bar{m}) \downarrow$. The dot marks the numeral substitution used to compute the added axiom’s code; x remains a free parameter of the consistency formula.

References

- [1] Kosta Došen and Zoran Petrić, *Proof-Theoretical Coherence*, King’s College Publications, London, 2004. Revised version, September 2007.
- [2] Saunders Mac Lane, *Categories for the Working Mathematician*, second edition, Graduate Texts in Mathematics 5, Springer, 1998.
- [3] F. William Lawvere, “Functorial semantics of algebraic theories,” *Proceedings of the National Academy of Sciences of the United States of America* **50** (1963), 869–872. doi:10.1073/pnas.50.5.869.
- [4] Olivia Caramello, *Topos Theory: Lectures 21 and 22—Classifying Toposes*. Lecture notes.
- [5] Graham Manuell, “The spectrum of a localic semiring,” *Mathematical Proceedings of the Cambridge Philosophical Society* **173** (2022), 647–668. doi:10.1017/S0305004122000068.
- [6] The Stacks Project Authors, “Points of schemes,” *The Stacks Project*. Tag 01J5.
- [7] Jacob Lurie, *Lecture 20X: Ultraproducts*, March 31, 2018. Lecture notes.
- [8] Peter J. Haine, *Classifying anima of condensed ∞ -categories of points*, arXiv:2602.21330, 2026.
- [9] Peter Scholze, *Lectures on Condensed Mathematics*, arXiv:2605.03658, 2026. Updated version of the 2019 lectures, presenting joint work with Dustin Clausen.
- [10] Solomon Feferman. *Arithmetization of metamathematics in a general setting*. *Fundamenta Mathematicae* **49** (1960), 35–92. doi:10.4064/fm-49-1-35-92.
- [11] Solomon Feferman. *Transfinite recursive progressions of axiomatic theories*. *The Journal of Symbolic Logic* **27**, no. 3 (1962), 259–316. doi:10.2307/2964649.
- [12] Robert A. Di Paola and Franco Montagna. *Some properties of the syntactic p -recursion categories generated by consistent, recursively enumerable extensions of Peano arithmetic*. *The Journal of Symbolic Logic* **56**, no. 2 (1991), 643–660. doi:10.2307/2274707.